LLM תכנון פיתוח מבבחן בית

https://gemini.google.com/app/910fa7dd35c567ff

*User prompt: במסגרת .1 AI בעזרת כלי LLM מבחן בית – פיתוח תוכנה מבוססת ////////////////////////////// AI בעזרת כלי LLM מבחן בית – פיתוח תוכנה מבוססת
על מנת לקצר את זמני הפיתוח ולשפר את התוצר הסופי. 1. יש להשלים את הפיתוח תוך 4 AI המבחן עלייך לפתח תוכנה איכותית בזמן קצר תוך הדגמת השימוש בכלי
שעות ולהגיש את התוצרים במייל. שעת התחלה 20:00 בערב. 1. יש להגיש את התוכנה המלאה, קוד המקור ותיעוד שיאפשר לבנות ולהריץ את התוכנה ברפו
פומבי. 1. ניתן לצרף כל מידע נוסף שיסייע בהבנת המערכת. 2. גם אם התוכנה לא רצה או מתפקדת יש להגיש את התוצר הטוב ביותר שהגעת אליו. 1. יש GITHUB
שלך. 1. עלייך להכיר את הקוד טוב מספיק כדי לענות על שאלות בעל פה ביחס לתוצר. 2. 3. חשוב! יש לספק את API -לשים לב שלא לספק עם הקוד את מפתחות ה
כחלק מתהליך הפיתוח, בכלל הכלים שבהם נעשה שימוש. 4. תהליך הפיתוח הינו חלק מהציון הסופי. יש לוודא שניתן לזהות את הבקשות AI כל השיחות שבוצעו עם
המשתמש בצורה ברורה ותשובות והפעולות של המודל. 5. 6. במידה ונשאר לך זמן רב מומלץ להרחיב את יכולות ואיכות התוכנה כך שהמוצר הסופי יהיה מרשים יותר.
לבחירתך. אין מגבלה לכלי ספציפי. 1. תיאור המשימה 1. עלייך לפתח תוכנה לבחינת תוכן של מסמכים בהתאם לסט AI 1. לצורך המשימה ניתן להשתמש בכל כלי
ומריץ ,DOCX חוקים מוגדר. 2. 3. ממשק המשתמש יהיה בדפדפן. 4. 5. המשתמש מגדיר חוקים טקסטואליים אותם יש לבדוק. 1. המשתמש מעלה קובץ לבחינה מסוג
את תהליך הבדיקה - בסופו יוצגו למשתמש מקרים בהם היו הפרות של החוקים. 1. לדוגמא: 1. המשתמש העלה מסמך המכיל תיאור מערכת מבצעית. 2. המשתמש
מבקש באחד החוקים שבמערכת, לבדוק האם יש במסמך תיאור מספרי כלשהו המסגיר את ביצועי המערכת. למשל, אם במסמך כתוב "טווח הגילוי המירבי הוא 50 ק"מ"
Agent Skill המערכת אמורה לזהות את ההפרה של החוק ותציג למשתמש את ההפרה. 1. המסמכים והחוקים נכתבים באנגלית בלבד. 1. ניקוד נוסף: יש לפתח
שמבצע את הבדיקה הזו וניתן לטעינה בכל סוכן סטנדרטי כגון Claude Code. בהצלחה !!! ////////////////////////////// we need to develop a solution to this
problem. Right now lets plan the solution step by step. 1) we will use docker, python, react, typescript, etc... 2) should we use a DB iin the
docker for this solution (for storing past rules) if yes we should allow the user a console to handle: adding rules, deleting rules, deactivatinf rules,
etc... 3) should we enable support for: docx, xslx, txt? 4) how do we handle files with a lot of pages/ sheets (if it is excel)? I think we should load
a 1 page at a time, extract the doument name and page and then send it to a model (page by page). But how can we handle cutted text for
model context? is it with chunking strategy? 5) should we use langraph? 6) we need to set up an LLM api key to use, where can I get one? I
have claude teams from my job and that is it (we can use open router if needed for the model). 7) how should we display the pages on the front?
I think about: file name, page number and the exact text (citation) next to the list of rules it defines. 8) If we use a DB, should we use alembic for
migrations?*

Response: This is a classic time-boxed architectural challenge. With only 4 hours, the biggest risk is overengineering. The goal is to deliver a robust MVP that perfectly hits the core requirements, with a clean codebase that shows seniority in GenAI systems.

Here is a breakdown of your 8 points to optimize for speed, quality, and the constraints of the test.

1. Stack (Docker, Python, React, TypeScript) This is the ideal stack.

- **Backend:** FastAPI. It's fast to set up, self-documenting (Swagger is great for exam submissions), and handles async LLM calls natively.
- **Frontend:** React with TypeScript and Vite. Use a component library like shadcn/ui or MUI to save styling time.
- **Infra:** A simple `docker-compose.yml` spinning up the frontend and backend containers.

2. Database & Rule Management Yes, use a database, but **use SQLite**. Do not spin up a separate PostgreSQL container—it adds points of failure and setup time.

- A simple SQLite file mounted to a Docker volume is perfect.
- Allowing the user to add, edit, and toggle rules (Active/Inactive) via a simple UI shows excellent product thinking.

3. File Support (DOCX, XLSX, TXT) The prompt explicitly asks for DOCX. **Focus only on DOCX for the MVP.** If you finish early, TXT is a trivial addition. Skip XLSX entirely unless you have abundant time; parsing spreadsheets into LLM-friendly context (handling empty cells, multiple sheets, and row/column headers) is a massive time sink and distracts from the core NLP task.

4. Document Handling & Context (Chunking vs. Full Context) For a 4-hour test, leverage modern large context windows rather than building a complex RAG pipeline.

- Extract the text using `python-docx`.
- If the document is reasonably sized, wrap the entire text in XML tags (e.g., `<document>...</document>`) and pass it to a model with a 128k+ context window. Ask the model to output a JSON array of violations, including the exact quote and the violated rule.
- **If you must chunk (for very large files):** Do not split strictly by page, as rules can span across page breaks. Use an overlapping text splitter (like `RecursiveCharacterTextSplitter`). When sending chunks to the LLM, include chunk metadata so the LLM can return the chunk ID for your frontend citation.

5. LangGraph Skip it. LangGraph is overkill for a straightforward text-evaluation pipeline and introduces unnecessary debugging complexity under a strict time limit. A direct async API call to the LLM (using the official SDKs or structured outputs) is faster, less error-prone, and easier for the reviewers to read.

6. LLM API Key You need an API key that supports structured JSON output and a decent context window.

- **OpenRouter:** Excellent choice. You can load it with a few dollars and access Claude 3.5 Sonnet or GPT-4o.
- **Google AI Studio:** You can get a free Gemini API key instantly, which gives you access to Gemini 1.5 Pro (massive context window, great at following complex rules).
- **Groq:** Free tier is available and incredibly fast, though context windows are smaller (e.g., Llama 3 70B).

7. Frontend Display Your proposed UX is spot on. A clean table or card layout showing:

- Rule Violated
- Exact Citation (The text from the document)
- Reasoning (A brief sentence from the LLM explaining *why* it violates the rule)
- *Tip:* Force the LLM to return structured JSON (using OpenAI's Structured Outputs or Gemini/Claude's JSON mode) matching this exact schema so your frontend can render it immediately without parsing logic.

8. Alembic Migrations Do not use Alembic. It consumes precious time to configure. Use SQLAlchemy or SQLModel and simply run `Base.metadata.create_all(bind=engine)` on application startup. For a take-home exam, this is universally accepted as the most efficient way to bootstrap the database.

Strategy for the "Agent Skill" Bonus

The prompt asks for an "Agent Skill" that can be loaded into a standard agent like **Claude Code**. The most modern and impressive way to do this is to expose your FastAPI backend as a **Model Context Protocol (MCP) Server**.

1. Build the core checking logic as a standard Python function.
2. Expose it via a FastAPI endpoint for your React app.
3. Add a simple MCP server script (using the `mcp` Python package) that exposes a `check_document_rules` tool. This proves you understand the cutting edge of multi-agent and CLI tool ecosystems.

User prompt: I don't want to pay for model calls I have google pro, so do yuo recommend getting the gemini API key?

Response: Yes, getting a Gemini API key is the absolute best move for this specific test. You do not need to pay anything to use it.

Your "Google Pro" (Google One AI Premium) subscription applies to the consumer web interface, but **Google AI Studio** offers a generous **Free Tier** for the API that is available to any Google account.

Here is why it fits your 4-hour constraint perfectly:

- **Zero Cost & No Credit Card:** You can generate an API key in seconds at Google AI Studio (aistudio.google.com) without setting up a billing account. It is completely free for development use.
- **Massive Context Window:** Gemini 1.5 Pro and Flash offer 1-to-2 million token context windows. You can extract the text from the DOCX file and feed the *entire* document to the model at once. This completely eliminates the need to write text chunking or pagination logic, saving you significant time.
- **Native Structured Outputs:** The Gemini API supports strict JSON schema outputs. You can define your exact Pydantic model (Rule Violated, Citation, Reasoning) and force Gemini to return exactly that structure, ensuring your frontend doesn't crash from malformed strings.
- **Rate Limits are Sufficient:** The free tier typically allows around 15 requests per minute, which is more than enough for developing, testing, and presenting your MVP.

How to implement it quickly:

1. Go to **Google AI Studio** -> Get API Key.
2. Install the SDK: `pip install google-genai` (the newest SDK) or use LangChain's integration if you prefer.
3. Pass your parsed DOCX string and your list of rules in the system prompt, requesting a JSON response mapping the violations.

User prompt: do I need any of this in my env file? or just the API key?

Response: You only need the API key.

For the standard Google AI Studio SDK, authentication relies entirely on the key itself. You can ignore the "Project name" and "Project number" fields shown in image_69f77b.png. Those are required only if you are deploying enterprise workloads via Google Cloud's Vertex AI platform.

Your `.env` file just needs this single line:

```
GEMINI_API_KEY=your_actual_api_key_string_here
```

User prompt: how do I keep track of all the conversation with claude (as the assignment wanted) becasue I will develop using calude code

Response: You can easily save your entire Claude Code conversation log using its built-in export command or by capturing your terminal session.

Here are the two best ways to ensure you have all your AI logs for the assignment submission.

Method 1: The Built-in `/export` Command (Recommended)

Claude Code has a native command to dump the current session's conversation into a readable markdown or text file.

1. Start your development session in your terminal by running `claude`.
2. Work on your assignment as usual, asking it questions and letting it generate code.
3. Before you close the session, type `/export` at the Claude Code prompt and press Enter.
4. An interactive menu will appear allowing you to either copy the conversation to your clipboard or save it to a file.
5. To skip the menu and save it directly, you can type `/export claude_log.md` (or any filename you prefer). This writes the full conversation as plain text directly to your project directory.

Verification: Open `claude_log.md` in your code editor to verify that both your prompts and Claude's responses (including tool outputs) are rendered as readable text.

Method 2: Terminal Transcript (Foolproof Backup)

If you want to guarantee that absolutely every keystroke and error is recorded just in case you forget to run the export command, use your operating system's native terminal logger before you launch Claude Code.

If you are on Mac or Linux:

1. Open your terminal and navigate to your project folder.
2. Type `script development_log.txt` and press Enter. (This starts recording everything on your screen).
3. Type `claude` to start Claude Code and do your work.
4. When finished, exit Claude Code (using `/quit` or `Ctrl+C`), then type `exit` and press Enter to stop the recording.

If you are on Windows (PowerShell):

1. Open PowerShell and navigate to your project folder.
2. Type `Start-Transcript -Path .\development_log.txt` and press Enter.
3. Type `claude` to start your session.
4. When finished, exit Claude, then type `Stop-Transcript` and press Enter.

Verification: Open `development_log.txt` to ensure the entire terminal session, including your interactions with Claude, was captured successfully.

User prompt: here is claude plan. waht do yu othink about the plan? - can luse an anthropic api key if i have claude teams from my work?

Response: This is an exceptionally strong, senior-level architectural plan. It directly addresses the constraints of the 4-hour take-home exam while anticipating edge cases (like DOCX pagination realities and LLM hallucinations) that most junior candidates would miss.

Your plan to use the Anthropic API directly is sound, but there is one major blocker based on your previous message: **You stated you do not want to pay for API calls.**

Here is an analysis of your plan, along with specific recommendations for the "Open Decisions" at the end, keeping your constraints in mind.

Plan Analysis & Strengths

- **Terminology Corrections (Section 2):** Pointing out that DOCX does not have fixed pages and utilizing heading paths instead shows deep domain knowledge of document parsing. This will impress the reviewers.
- **Chunking vs. Full Context (Section 4/5):** While my previous advice suggested full-context to save time, your chunking strategy is highly sophisticated. Overlapping blocks and deduplication are production-grade patterns for exhaustive search tasks. If you can implement this quickly, do it.
- **Verification (Anti-Hallucination) (Section 5.4):** This is a standout feature. Forcing a string-match verification before showing the quote to the user proves you understand the unreliability of LLMs in production.
- **Skill vs. MCP (Section 8):** Your distinction is correct. If the prompt explicitly asks for an "Agent Skill" compatible with Claude Code, a `SKILL.md` is the precise answer.

Answering Your Open Decisions

1. Provider: Anthropic API vs. Gemini (Crucial Pivot)

- *Your constraint:* You do not want to pay for API calls, and your work "Claude Teams" account does not provide an API key.
- *The Reality:* You cannot use the Anthropic API without attaching a credit card and buying prepaid credits on the Anthropic Console. OpenRouter also requires prepaid credits for Claude models.
- *The Solution:* **You must switch back to the Gemini API (Google AI Studio).**
  - As discussed, you can get a Gemini API key for free instantly.
  - The `google-genai` SDK supports native Structured Outputs (JSON Schema) just like Anthropic.
  - Gemini 1.5 Pro is highly capable of following your system prompts and extracting the violations.
  - *Action Item:* Update Section 3.Q6 and Section 5.3 of your plan to reflect the `google-genai` SDK and Gemini 1.5 Pro.

2. Default Model

- Given the pivot above, your default model should be `gemini-1.5-pro`. It has a massive context window and excels at complex reasoning and needle-in-a-haystack retrieval.

3. Check history in the DB: MVP or stretch?

- **Stretch.** Do not attempt this until the core pipeline is completely finished and the Docker container builds successfully. Adding history requires a new table, new API endpoints, and a new UI view. It is a time-sink that distracts from the core requirement: running the check.

4. Keep `CLAUDE.md` out of the repo?

- **Keep it in the repo (Modify `.gitignore`).** The exam instructions explicitly state: *"חשוב! יש לספק את כל השיחות שבוצעו עם AI כחלק מתהליך הפיתוח, בכלל הכלים שבהם נעשה שימוש. תהליך הפיתוח הינו חלק מהציון הסופי."* (Important! You must provide all conversations conducted with AI...). The development process is part of the final grade.
- Including `CLAUDE.md` and your exported conversation logs proves exactly how you guided the AI. It is a feature of your submission, not a bug.

Final Verdict on the Plan

The plan is excellent (a solid 9.5/10 for a senior architecture).

Your immediate next step: Modify the plan to use the **Gemini API** via Google AI Studio so you can adhere to your "no cost" constraint, generate your free API key, and begin the Scaffold phase (20:00 - 20:20).

User prompt: what parameter do I need in order to use the gemini free model in my env except for the api key

Response: You do not need any other parameters for authentication. Because you are using the Google AI Studio API and not Google Cloud Vertex AI, you do not need a Project ID, Region, or Service Account JSON.

However, to follow the clean architecture outlined in your plan (specifically keeping configurations out of hardcoded logic), you should add the **model name** to your environment file.

Your `.env` file should look exactly like this:

```
GEMINI_API_KEY=your_actual_api_key_string_here
GEMINI_MODEL=gemini-1.5-pro
```

Why this is all you need

When you use the official `google-genai` Python SDK, the client automatically searches your environment for `GEMINI_API_KEY`. You initialize it instantly with zero configuration:

```
import os
from google import genai

# Automatically picks up GEMINI_API_KEY from the environment
client = genai.Client()

# Pull the model name from your .env so it isn't hardcoded
model_name = os.getenv("GEMINI_MODEL", "gemini-1.5-pro")

response = client.models.generate_content(
    model=model_name,
    contents="Your extracted DOCX text here"
)
```

Recommendation for your test: Use `gemini-1.5-pro`. It has a 2-million token context window on the free tier, which is essential for your strategy of passing large document chunks alongside complex validation rules.

User prompt: are you sure gemini 1.5 pro is still available and that your information is up to date

Response: You are exactly right to question that—my previous recommendation was outdated for our current timeline (late 2026).

Here is the up-to-date reality for Google AI Studio's free tier and available models:

- **Free Tier Restrictions:** As of April 1, 2026, Google removed all "Pro" models from the AI Studio free tier. Currently, only "Flash" and "Flash-Lite" models retain free access, which includes a rate limit of 5 to 15 requests per minute.
- **Model Deprecation:** Gemini 1.5 Pro is severely outdated. Even the much newer Gemini 2.5 Pro is now considered deprecated and will automatically route to newer models by December 2026.
- **The 2026 Recommendation:** You should update your plan to use `gemini-3.8-flash`. Released in September 2026, it is Google's current primary Flash model. It is available on the free tier and maintains a 1-million token context window, which perfectly supports your strategy of passing large, overlapping document chunks.

Your `.env` file should be updated to reflect the modern model:

```
GEMINI_API_KEY=your_actual_api_key_string_here
GEMINI_MODEL=gemini-3.8-flash
```

Since the free tier limits you to 5-15 requests per minute, your chunking strategy (which groups document sections rather than firing a request for every single sentence) is actually even more critical now to avoid hitting rate limits while keeping the pipeline fast.

User prompt: Decisions I need from you (also listed at the end of the plan): 1. Which provider: Anthropic directly (my recommendation) or OpenRouter? 2. Which default model: claude-opus-5-5, or a cheaper Sonnet 5.5 or Haiku 5.5? 3. Should saving past check results be in the first version, or left as a stretch goal? 4. Should CLAUDE.md and .claude/ go into the public repo as evidence of how the AI was guided? ///////// answer claude open questions like this is my response

Response: We are making a pivot on the provider to ensure we use a free API. Here are the decisions to lock in the plan:

1. **Provider**: Neither Anthropic nor OpenRouter. We will use the **Gemini API (Google AI Studio)** via the official `google-genai` Python SDK, as it provides a free tier that fits this project perfectly.
2. **Default model**: Use `gemini-1.5-flash` (or `gemini-1.5-pro` depending on current free tier availability). Configure this via a `GEMINI_MODEL` environment variable.
3. **Check history**: Leave saving past check results as a stretch goal. Given the strict 4-hour timebox, we must prioritize building a robust core pipeline (extraction, chunking, verification, and UI) first.
4. **AI evidence**: Yes, include `CLAUDE.md`, the `.claude/` directory, and any exported conversation logs in the public repo. You must modify `.gitignore` so these files are tracked. The exam instructions explicitly require submitting all AI conversations and proof of the development process for the final grade.

User prompt: but you said gemini 1.5 is not available nymore

Response: You caught me—I apologize for the slip-up. I accidentally reverted to the outdated model names in that bullet point right after explaining the 2026 updates to you.

You are completely right: the 1.5 and 2.5 models are deprecated or actively being phased out in late 2026.

Here is the corrected decision for your list:

2. **Default model:** Use **`gemini-3.8-flash`** (or `gemini-3.5-flash-lite`). Both are active and available on the Google AI Studio free tier. Configure this via the `GEMINI_MODEL` environment variable.

The rest of the decisions from the previous message remain correct. Go ahead and update your plan with `gemini-3.8-flash` and begin your development.

User prompt: HOW SHOUSL WE HANDLE OUT packages na d versions? using requirements.txt, .toml or other file that wil lgo with docker

Response: Use `pyproject.toml` managed by `uv`, generating a `uv.lock` file for deterministic builds.

This approach aligns perfectly with the modern Python stack outlined in your architecture plan and ensures your exact local environment is replicated in Docker without the slowness of standard `pip`.

Here is the exact workflow to implement this for your 4-hour test:

1. **Local Setup:** Initialize your dependencies using `uv add fastapi google-genai pydantic python-docx`. This automatically updates your `pyproject.toml` and generates a `uv.lock` file.
2. **Docker Strategy:** Do not manually maintain a `requirements.txt`. Instead, use the official `ghcr.io/astral-sh/uv` Docker image as a builder, or copy the lockfile and use `uv sync` directly inside your container.

Here is the most efficient Dockerfile pattern to use with this setup:

```
FROM python:3.12-slim

# Install uv
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/

WORKDIR /app

# Copy dependency files first to leverage Docker layer caching
COPY pyproject.toml uv.lock ./

# Install dependencies into the system environment (avoids virtualenv overhead in Docker)
RUN uv sync --frozen --no-cache --system

# Copy the rest of the application
COPY src ./src

CMD ["fastapi", "run", "src/main.py", "--port", "8080", "--host", "0.0.0.0"]
```

This guarantees your builds are reproducible for the exam reviewers while keeping your build times under a few seconds.

User prompt: The plan is final: knowledge/plans/01-solution-plan.md. I haven't implemented anything. Two corrections to your choices: - gemini-1.5-pro is no longer offered, so it can't be the fallback. I used gemini-2.5-flash instead. gemini-3.8-flash stays the main model, and both are listed as free on Google's pricing page. - The free tier allows very few requests. Limits are counted per model, and some reports put Flash models at about 20 requests per day. Each chunk of the document is one request, so the plan now: - uses large chunks (about 30k tokens, adjustable), so most documents take 1 to 3 requests; - switches to the fallback model automatically when the main one hits its daily

limit; - shows a clear "limit reached" message in the UI; - runs mocked tests during development and keeps real API calls for the smoke test and the final demo. What changed from the first draft: - Gemini: calls go through the google-genai SDK, and the model is told to return JSON matching a fixed schema. The key is GEMINI_API_KEY in .env. - Safety blocks: Gemini may block text about military systems. The plan treats a blocked chunk as failed and marks the whole check as partial, never as "no violations". - Free-tier data use: on the free tier, Google may use the uploaded content to improve its products. The README will say to use only made-up sample documents. - Check history is in the first version. Each run stores the document name, a fingerprint of the file, the model used, a copy of the rules as they were at that moment, and the results. It does not store the uploaded file. There are history endpoints and a History page. - .claude/ and CLAUDE.md: this is the last step on the checklist. Stop ignoring them in .gitignore, keep settings.local.json and apm_modules/ ignored, and check for secrets before committing. - Still to confirm when you start: the exact model IDs your key can use (Google sometimes adds a "preview" suffix) and your current daily limits in the AI Studio dashboard. I also saved a short note about this project and your choices to memory for future sessions. Run /export at the end of this session, because the exam requires submitting every AI conversation. Sources: - Gemini API rate limits - Gemini API pricing - Forum: free-tier quotas counted per model - Forum: Gemini 3.8 Flash free tier at 20 requests per day ✻ Baked for 1m 56s · done 7:38 PM ❯ gemini-2.5-flash instead. gemini-3.8-flash, the main model and fallnack shoud be loaded using hte .env file, so tell me waht variables to put there. We will use pyproject.toml and uv for packages. keep in mind that we have skills for python, frontend and tdd and they should be read before the development. Also we will develop using tdd approach ● Update(knowledge\plans\01-solution-plan.md) └ Added 2 lines 19 12. [Submission Checklist](#12-submission-checklist) 20 13. [Risks and Mitigations](#13-risks-and-mitigations) 21 14. [Verify at Implementation Time](#14-verify-at-implementation-time) 22 +15. [Configuration (.env)](#15-configuration-env) 23 +16. [Development Workflow: Skills First, TDD Always](#16-development-workflow-skills-first-tdd-always) 24 25 --- 26 ● Update(knowledge\plans\01-solution-plan.md) └ Added 1 line, removed 1 line 61 62 ### Q1. Stack 63 64 -- Backend: Python 3.12, **FastAPI**, Pydantic v2, SQLModel, `uv` + `ruff` + `ty`, `pytest`. 64 +- Backend: Python 3.12, **FastAPI**, Pydantic v2, SQLModel, pydantic-settings, `pytest`. Packaging: **`pyproject.toml` managed with `uv`** (`uv add`, `uv sync`, committed `uv.l +ock`), linting with `ruff`, type checking with `ty`. No `requirements.txt`. 65 - Frontend: **Vite + React + TypeScript**, Tailwind, TanStack Query, React Hook Form + Zod, React Router, Vitest. 66 - Infra: `docker-compose.yml` with `backend` and `frontend` services. The frontend is built and served by nginx, which reverse-proxies `/api` to the backend (single origin, no CORS setup). 67 ● Update(knowledge\plans\01-solution-plan.md) └ Added 1 line, removed 1 line 100 **Google Gemini API (Google AI Studio key) via the official `google-genai` Python SDK.** 101 102 - Key: create in Google AI Studio (`aistudio.google.com` -> Get API key). Stored as `GEMINI_API_KEY` in `.env` (already gitignored), placeholder in `.env.example`, passed to t he backend via `env_file`. Never logged. 103 -- **Primary model: `gemini-3.8-flash`**, set by env var `LLM_MODEL`. **Fallback: `gemini-2.5-flash`** (`LLM_FALLBACK_MODEL`), used automatically when the primary returns a quo -ta error (429), because free-tier quotas are counted per model. Both are listed as free-tier models on the Gemini pricing page. 103 +- **Primary model: `gemini-3.8-flash`** (`LLM_MODEL`). **Fallback: `gemini-2.5-flash`** (`LLM_FALLBACK_MODEL`), used automatically when the primary returns a quota error (429) +, because free-tier quotas are counted per model. Both are listed as free-tier models on the Gemini pricing page. **Both model names are read only from `.env`** (no defaults i +n code); see [section 15](#15-configuration-env). 104 - **Structured output:** `response_mime_type="application/json"` + `response_schema=<Pydantic model>`, read via `response.parsed`. Temperature 0 for repeatability. System prom pt via `system_instruction`. 105 - Async calls via `client.aio.models.generate_content`. 106 - **Safety blocks:** Gemini may block a response (`finish_reason == SAFETY`) on defense-related content. Treated as a failed chunk, surfaced to the user, never as "no violatio ns". ● Update(knowledge\plans\01-solution-plan.md) └ Added 3 lines, removed 2 lines 372 373 | Time | Block | Output | 374 |---|---|---| 375 -| 20:00 to 20:20 | Scaffold | Backend (uv, FastAPI, ruff, pytest), frontend (Vite), compose, `.env.example`, README skeleton; first push to the public repo | 376 -| 20:20 to 21:05 | Core pipeline, test-first | Extractors (DOCX, TXT), chunker, verifier + fixture DOCX | 375 +| 20:00 to 20:10 | Read skills | Python + TDD skills (section 16) | 376 +| 20:10 to 20:25 | Scaffold | Backend (`pyproject.toml` via `uv`, FastAPI, ruff, ty, pytest), frontend (Vite), compose, `.env.example`, README skeleton; first push to the publ +ic repo | 377 +| 20:25 to 21:05 | Core pipeline, test-first | Extractors (DOCX, TXT), chunker, verifier + fixture DOCX | 378 | 21:05 to 21:35 | LLM integration | `LlmClient` + Gemini implementation (schema, fallback, safety handling), `ComplianceChecker`, real-API smoke test on the sample | 379 | 21:35 to 22:00 | API + DB | Rules CRUD, seed, `/api/checks` with persistence, history endpoints, API tests | 380 | 22:00 to 22:50 | Frontend | Check, Rules and History pages, results view | ● Update(knowledge\plans\01-solution-plan.md) └ Added 1 line, removed 1 line 377 | 20:25 to 21:05 | Core pipeline, test-first | Extractors (DOCX, TXT), chunker, verifier + fixture DOCX | 378 | 21:05 to 21:35 | LLM integration | `LlmClient` + Gemini implementation (schema, fallback, safety handling), `ComplianceChecker`, real-API smoke test on the sample | 379 | 21:35 to 22:00 | API + DB | Rules CRUD, seed, `/api/checks` with persistence, history endpoints, API tests | 380 -| 22:00 to 22:50 | Frontend | Check, Rules and History pages, results view | 380 +| 22:00 to 22:50 | Frontend | Read frontend skills first (section 16), then Check, Rules and History pages, results view, test-first for schemas and hooks | 381 | 22:50 to 23:15 | Docker end-to-end | `docker compose up --build` from a clean clone | 382 | 23:15 to 23:40 | Agent Skill | `SKILL.md`, `extract.py`, `verify.py`, tried in Claude Code | 383 | 23:40 to 24:00 | Submission | README, un-ignore `.claude/` + `CLAUDE.md`, export AI conversations, final push, email | ● Update(knowledge\plans\01-solution-plan.md) └ Added 53 lines 431 - `google-genai` structured-output and async call signatures against the SDK docs before writing the client. 432 433 Sources: [Gemini API rate limits](https://ai.google.dev/gemini-api/docs/rate-limits), [Gemini API pricing](https://ai.google.dev/gemini-api/docs/pricing). 434 + 435 +--- 436 + 437 +## 15. Configuration (.env) 438 + 439 +Loaded by `pydantic-settings` in `config.py`. **Required** variables have no default: the backend refuses to start and names the missing variable (fail fast). **Optional** var +iables override defaults that live as named constants in `constants.py`. 440 + 441 +| Variable | Required | Example / default | Purpose | 442 +|---|---|---|---| 443 +| `GEMINI_API_KEY` | yes | `AIza...` (your key) | Google AI Studio key | 444 +| `LLM_MODEL` | yes | `gemini-3.8-flash` | Primary model | 445 +| `LLM_FALLBACK_MODEL` | yes | `gemini-2.5-flash` | Used when the primary hits its quota (429) | 446 +| `LLM_TEMPERATURE` | no | `0` | Repeatable output | 447 +| `LLM_MAX_CONCURRENCY` | no | `2` | Parallel chunk requests (free-tier RPM) | 448 +| `CHUNK_TOKEN_BUDGET` | no | `30000` | Max estimated tokens per chunk | 449 +| `CHUNK_OVERLAP_BLOCKS` | no | `2` | Blocks shared between consecutive chunks | 450 +| `MAX_UPLOAD_MB` | no | `20` | Upload size limit | 451 +| `DATABASE_URL` | no | `sqlite:////data/doc_checker.db` | SQLite file inside the Docker volume | 452 +| `LOG_LEVEL` | no | `INFO` | Logging level | 453 + 454 +`.env.example` (committed) holds the same keys with `GEMINI_API_KEY=` left empty. `.env` (gitignored) holds the real key. Docker Compose passes it with `env_file: .env`. 455 + 456 +---

457 +
458 +## 16. Development Workflow: Skills First, TDD Always
459 +
460 +### Read before writing code *461 +* *462 +| When | Skills to read | What they decide |* *463 +|---|---|---|* *464 +| Before backend scaffold | `python-tooling` | `uv`, `pyproject.toml`, ruff, ty configuration |* *465 +| Before backend scaffold | `python-layout` | src layout, where constants vs config live, file splitting |* *466 +| Before backend code | `python-patterns` | dataclass vs Pydantic, enums, error handling, logging, function design |* *467 +| Before every backend feature | `tdd-workflow` | interface first, failing tests, implementation, refactor |* *468 +| Before frontend scaffold | `web-stack` | Vite/React/TS stack, feature-sliced structure, repository/hooks/schemas split, Vitest |* *469 +| Before frontend UI work | `ui-building` | component and page conventions |* *470 +| Before Dockerfiles | `container-deployment` | Dockerfile conventions |* *471 +* *472 +If a skill conflicts with this plan, the skill wins and this plan is updated.* *473 +* *474 +### TDD cycle (every feature and bug fix)* *475 +* *476 +1. **Interface:** define the protocol / function signature and Pydantic models, no logic.* *477 +2. **Red:** write tests from the success criteria in section 9; run them and confirm they fail for the right reason.* *478 +3. **Green:** minimal implementation until the tests pass.* *479 +4. **Refactor:** clean up with tests green; run `ruff check`, `ruff format`, `ty check`.* *480 +* *481 +Rules:* *482 +- Mock only at boundaries: `LlmClient` (never the Gemini SDK internals), the DB via an in-memory SQLite session.* *483 +- Fixture DOCX files are generated by a test helper with `python-docx` and committed under `tests/fixtures/`.* *484 +- Real Gemini calls only in the opt-in integration test (`pytest -m integration`), to save free-tier quota.* *485 +- Frontend: Vitest for Zod schemas, API repository functions and hooks before the components that use them.* *486 +- Feature order follows the dependency chain: extraction, chunking, verification, LLM client, service, repository, API, frontend.* *Wrote 1 memory (ctrl+o to expand) ● I've updated the plan in knowledge/plans/01-solution-plan.md. I didn't touch your existing .env or .env.example. Your .env dotenv # Required: the backend will refuse to start and name the missing variable GEMINI_API_KEY=your-key-from-aistudio.google.com LLM_MODEL=gemini-3.8-flash LLM_FALLBACK_MODEL=gemini-2.5-flash # Optional: these values are the defaults; set only to override LLM_TEMPERATURE=0 LLM_MAX_CONCURRENCY=2 CHUNK_TOKEN_BUDGET=30000 CHUNK_OVERLAP_BLOCKS=2 MAX_UPLOAD_MB=20 DATABASE_URL=sqlite:////data/doc_checker.db LOG_LEVEL=INFO - Model names: both are read only from .env, with no defaults in code, so swapping models never needs a code change. - .env.example: gets the same variables with GEMINI_API_KEY= left empty. It's committed; .env stays gitignored. Changes to the plan: - Packages: pyproject.toml managed with uv, and uv.lock is committed. There's no requirements.txt. - New section 15 lists the configuration variables above. - New section 16, skills before code: | When | Skills to read | |------------------------|--| | Before the backend | python-tooling, python-layout, python-patterns, tdd-workflow | | Before the frontend | web-stack, ui-building | | Before the Dockerfiles | container-deployment | If a skill disagrees with the plan, the skill wins and I'll update the plan. - TDD cycle, also in section 16: for every feature, define the interface, write tests that fail, implement until they pass, then clean up. The tests come from the success criteria in section 9. - Only the LLM client and the database are replaced with fakes in tests. - Real Gemini calls run only when you ask for them with pytest -m integration, to save the free-tier quota. - Features are built in dependency order: extraction, chunking, verification, LLM client, service, repository, API, frontend. - Time plan: the first 10 minutes (20:00 to 20:10) are for reading the Python and TDD skills. The frontend skills are read at the start of the frontend block. The plan is final. I'll wait for your go before starting development.*

Response: The architecture is fully locked in and production-ready. The explicit handling of safety blocks for operational system descriptions and the strict verification layer for hallucinated citations are exactly the kind of robust edge-case handling expected from a senior AI Engineer in the defense sector.

Give Claude the green light to execute the scaffold and start the 20:00 - 20:10 block to read the python and TDD skills.

As you move through the sprint, keep a close eye on the `google-genai` SDK integration during the 21:05 block. If the model struggles to strictly adhere to the Pydantic schema for the violation reports, or if the 30k token chunks start causing unexpected safety refusals on benign system specs, pause and drop the errors here so we can tighten the system instructions.

Start the clock and kick off the first TDD cycle. I am on standby if you hit any Docker networking snags or free-tier rate-limit anomalies.

User prompt: if I wantr to add poe command (uv run poe ...) where is the correct file to add tehm to

Response: Add your `poe` commands directly to your **`pyproject.toml`** file under the `[tool.poe.tasks]` section.

Here is how you format it:

```
[tool.poe.tasks]
dev = "fastapi run src/main.py --port 8080 --reload"
test = "pytest"
lint = "ruff check ."
format = "ruff format ."
```

Since you are already using `pyproject.toml` to manage your stack with `uv`, this keeps all your project dependencies, metadata, and task scripts unified in a single file. You can then run them exactly as you noted: `uv run poe dev` or `uv run poe test`.

User prompt: ❯ start implementing the backend scaffold
● Skill(python-tooling) ⎿ Successfully loaded skill ● Skill(python-layout) ⎿ Successfully loaded skill ● Skill(python-patterns) ⎿ Successfully loaded skill ● Skill(tdd-workflow) ⎿ Successfully loaded skill ● Skill(container-architecture) ⎿ Successfully loaded skill ● Bash(uv --version; python --version; docker --version; cat .env.example; git status --short; cat README.md) ⎿ uv 0.11.21 (5aa65dd7a 2026-

06-11 x86_64-pc-windows-msvc) Python 3.13.13 Docker version 29.9.2, build a82e528 … +5 lines (ctrl+o to expand) └ Allowed by auto mode ● Bash(cut -d= -f1 .env; uv python list --only-installed 2>&1 | head) └ GEMINI_API_KEY GEMINI_MODEL error: Failed to inspect Python interpreter from search path at `C:\Users\shilo\.local\bin\python3.14.exe` … +2 lines (ctrl+o to expand) └ Allowed by auto mode ● The layout skill is a batch-job pattern that doesn't fit this long-running HTTP API, so I'll use the plain src/ layout and borrow only the fail-fast config and startup wiring. Python 3.14 is blocked here by policy, so I'm pinning 3.13, and the .env uses GEMINI_MODEL where the plan needs LLM_MODEL and LLM_FALLBACK_MODEL. Starting with the manifest and tests. · summary ● Bash(mkdir -p /c/projects/elta_hw/backend/src/doc_checker/api /c/projects/elta_hw/backend/src/doc_checker/custom_types /c/projects/elta_hw/backend/tests/unit && echo…) └ total 5 drwxr-xr-x 1 shilo 197609 0 Nov 10 19:44 . drwxr-xr-x 1 shilo 197609 0 Nov 10 19:44 .. … +3 lines (ctrl+o to expand) └ Created backend\.python-version (+1 -0) 1 +3.13 └ (files that changed while this command ran, by the command or by another process; a convenience view, not a review or audit of the command) ● Write(backend\pyproject.toml) └ Wrote 24 lines to backend\pyproject.toml 1 [project] 2 name = "doc-checker" 3 version = "0.1.0" 4 description = "LLM-based document rule compliance checker (backend API)" 5 requires-python = ">=3.13" 6 dependencies = [] 7 8 [build-system] 9 requires = ["setuptools>=68"] 10 build-backend = "setuptools.build_meta" … +14 lines (ctrl+o to expand) ● Bash(cd /c/projects/elta_hw/backend && uv add fastapi "uvicorn[standard]" pydantic-settings 2>&1 | tail -5 && uv add --dev pytest ruff ty httpx 2>&1 | tail -5 && cat…) └ + typing-extensions==4.16.0 + typing-inspection==0.4.5 + uvicorn==0.54.0 … +27 lines (ctrl+o to expand) └ Updated pyproject.toml (+13 -1) 3 version = "0.1.0" 4 description = "LLM-based document rule compliance checker (backend API)" 5 requires-python = ">=3.13" 6 -dependencies = [] 6 +dependencies = [7 + "fastapi>=0.162.3", 8 + "pydantic-settings>=2.16.2", 9 + "uvicorn[standard]>=0.54.0", 10 +] 11 12 [build-system] 13 requires = ["setuptools>=68"] ... 26 testpaths = ["tests"] 27 markers = ["integration: calls the real Gemini API (needs GEMINI_API_KEY)"] 28 addopts = "-m 'not integration'" 29 + 30 +[dependency-groups] 31 +dev = [32 + "httpx>=0.28.1", 33 + "pytest>=9.2.0", 34 + "ruff>=0.17.4", 35 + "ty>=0.0.100", 36 +] └ Created src\doc_checker.egg-info\PKG-INFO (+8 -0) 1 +Metadata-Version: 2.4 2 +Name: doc-checker 3 +Version: 0.1.0 4 +Summary: LLM-based document rule compliance checker (backend API) 5 +Requires-Python: >=3.13 6 +Requires-Dist: fastapi>=0.162.3 7 +Requires-Dist: pydantic-settings>=2.16.2 8 +Requires-Dist: uvicorn[standard]>=0.54.0 └ Created src\doc_checker.egg-info\SOURCES.txt (+6 -0) 1 +pyproject.toml 2 +src/doc_checker.egg-info/PKG-INFO 3 +src/doc_checker.egg-info/SOURCES.txt 4 +src/doc_checker.egg-info/dependency_links.txt 5 +src/doc_checker.egg-info/requires.txt 6 +src/doc_checker.egg-info/top_level.txt └ Created src\doc_checker.egg-info\dependency_links.txt (+1 -0) 1 + └ Created src\doc_checker.egg-info\requires.txt (+3 -0) 1 +fastapi>=0.162.3 2 +pydantic-settings>=2.16.2 3 +uvicorn[standard]>=0.54.0 └ … 2 more files changed └ (timeout 5m) └ Allowed by auto mode ● Dependencies are in. Now red: the tests for the scaffold's three units (config, JSON logging, health endpoint), written before any implementation. ● Write(tests\conftest.py) └ Wrote 31 lines to tests\conftest.py 1 from __future__ import annotations 2 3 import pytest 4 5 from doc_checker.config import AppConfig 6 7 REQUIRED_ENV = { 8 "GEMINI_API_KEY": "test-key-not-real", 9 "LLM_MODEL": "test-primary-model", 10 "LLM_FALLBACK_MODEL": "test-fallback-model", … +21 lines (ctrl+o to expand) ● Write(tests\unit\test_config.py) └ Wrote 83 lines to tests\unit\test_config.py 1 from __future__ import annotations 2 3 import pytest 4 from pydantic import ValidationError 5 6 from doc_checker.config import AppConfig 7 from doc_checker.custom_types.config_types import LogLevel 8 9 10 def test_loads_required_values_and_defaults(required_env: pytest.MonkeyPatch) -> None: … +73 lines (ctrl+o to expand) ● Write(tests\unit\test_logging_config.py) └ Wrote 43 lines to tests\unit\test_logging_config.py 1 from __future__ import annotations 2 3 import json 4 import logging 5 6 from doc_checker.logging_config import JsonFormatter 7 8 9 def test_formats_record_as_json_with_standard_and_extra_fields() -> None: 10 record = logging.makeLogRecord(… +33 lines (ctrl+o to expand) ● Write(tests\unit\test_health_router.py) └ Wrote 27 lines to tests\unit\test_health_router.py 1 from __future__ import annotations 2 3 from fastapi.testclient import TestClient 4 5 from doc_checker.config import AppConfig 6 from doc_checker.main import create_app 7 8 9 def test_health_returns_ok_and_configured_models(app_config: AppConfig) -> None: 10 client = TestClient(create_app(app_config)) … +17 lines (ctrl+o to expand) ● Write(tests\unit\test_logging_config.py) └ Added 2 lines, removed 3 lines 2 3 import json 4 import logging 5 +import sys 6 7 from doc_checker.logging_config import JsonFormatter 8 ... 34 try: 35 raise RuntimeError("boom") 36 except RuntimeError: 36 - record = logging.makeLogRecord({"msg": "failed", "exc_info": True}) 37 - import sys 38 - 37 + record = logging.makeLogRecord({"msg": "failed"}) 38 record.exc_info = sys.exc_info() 39 40 payload = json.loads(JsonFormatter("doc-checker").format(record)) ● Bash(touch tests/__init__.py tests/unit/__init__.py && uv run pytest -q 2>&1 | tail -8) └ ImportError while loading conftest 'C:\projects\elta_hw\backend\tests\conftest.py'. tests\conftest.py:5: in <module> from doc_checker.config import AppConfig E ModuleNotFoundError: No module named 'doc_checker.config' └ Created tests__init__.py (+0 -0) └ Created tests\unit__init__.py (+0 -0) └ … 2 more files changed └ Allowed by auto mode ● Red, for the right reason: the modules don't exist yet. Now green: ● Write(src\doc_checker__init__.py) ● Write(src\doc_checker\api__init__.py) ● Write(src\doc_checker\api__init__.py) 1 (No content) ● Write(src\doc_checker\custom_types__init__.py) └ Wrote 1 line to src\doc_checker\custom_types__init__.py 1 (No content) ● Write(src\doc_checker\constants.py) └ Wrote 8 lines to src\doc_checker\constants.py 1 from __future__ import annotations 2 3 APPLICATION_NAME = "doc-checker" 4 APP_TITLE = "Document Rule Compliance Checker" 5 6 API_PREFIX = "/api" 7

HEALTH_PATH = "/health" 8 HEALTH_STATUS_OK = "ok" ● Write(src\doc_checker\custom_types\config_types.py) └ Wrote 10 lines to src\doc_checker\custom_types\config_types.py 1 from __future__ import annotations 2 3 from enum import StrEnum 4 5 6 class LogLevel(StrEnum): 7 DEBUG = "DEBUG" 8 INFO = "INFO" 9 WARNING = "WARNING" 10 ERROR = "ERROR" ● Write(src\doc_checker\config.py) └ Wrote 40 lines to src\doc_checker\config.py 1 from __future__ import annotations 2 3 from pydantic import Field, SecretStr, field_validator 4 from pydantic_settings import BaseSettings, SettingsConfigDict 5 6 from doc_checker.custom_types.config_types import LogLevel 7 8 9 class AppConfig(BaseSettings): 10 """Environment-driven configuration. Created once at startup and passed down. … +30 lines (ctrl+o to expand) ● Write(src\doc_checker\logging_config.py) └ Wrote 36 lines to src\doc_checker\logging_config.py 1 from __future__ import annotations 2 3 import json 4 import logging 5 import sys 6 from datetime import UTC, datetime 7 8 # Attributes every LogRecord carries; anything else on a record came from `extra={}`. 9 _RESERVED_ATTRS = frozenset(vars(logging.makeLogRecord({}))) | {"message", "asctime", "taskName"} … +27 lines (ctrl+o to expand) ● Write(src\doc_checker\api\dependencies.py) └ Wrote 9 lines to src\doc_checker\api\dependencies.py 1 from __future__ import annotations 2 3 from fastapi import Request 4 5 from doc_checker.config import AppConfig 6 7 8 def get_config(request: Request) -> AppConfig: 9 return request.app.state.config ● Write(src\doc_checker\api\health_router.py) └ Wrote 29 lines to src\doc_checker\api\health_router.py 1 from __future__ import annotations 2 3 from typing import Annotated 4 5 from fastapi import APIRouter, Depends 6 from pydantic import BaseModel, ConfigDict 7 8 from doc_checker.api.dependencies import get_config 9 from doc_checker.config import AppConfig 10 from doc_checker.constants import HEALTH_PATH, HEALTH_STATUS_OK … +19 lines (ctrl+o to expand) ● Write(src\doc_checker\main.py) └ Wrote 30 lines to src\doc_checker\main.py 1 from __future__ import annotations 2 3 import logging 4 5 from fastapi import FastAPI 6 7 from doc_checker.api.health_router import router as health_router 8 from doc_checker.config import AppConfig 9 from doc_checker.constants import API_PREFIX, APP_TITLE, APPLICATION_NAME 10 from doc_checker.logging_config import configure_logging … +20 lines (ctrl+o to expand) ● Bash(uv sync -q 2>&1 | tail -3; uv run pytest -q 2>&1 | tail -15) └ ..... [100%] ========== warnings summary ========== .venv\Lib\site-packages\fastapi\testclient.py:1 … +6 lines (ctrl+o to expand) └ 14 files changed (binary, mode only or too large to show) └ (timeout 3m) └ Allowed by auto mode ● All 20 green. The warning says Starlette now wants httpx2, so I'll swap that, then run the refactor gate (ruff + ty). ● Bash(uv remove --dev httpx -q 2>&1 | tail -2; uv add --dev httpx2 -q 2>&1 | tail -2; uv run pytest -q 2>&1 | tail -3; echo "--- ruff"; uv run ruff check . ; uv run r…) └ ..... [100%] 20 passed in 0.31s --- ruff … +23 lines (ctrl+o to expand) └ Updated pyproject.toml (+1 -1) 29 30 [dependency-groups] 31 dev = [32 - "httpx>=0.28.1", 32 + "httpx2>=2.16.0", 33 "pytest>=9.2.0", 34 "ruff>=0.17.4", 35 "ty>=0.0.100", └ Updated src\doc_checker.egg-info\SOURCES.txt (+11 -1) 1 pyproject.toml 2 +src/doc_checker/__init__.py 3 +src/doc_checker/config.py 4 +src/doc_checker/constants.py 5 +src/doc_checker/logging_config.py 6 +src/doc_checker/main.py 7 src/doc_checker.egg-info/PKG-INFO 8 src/doc_checker.egg-info/SOURCES.txt 9 src/doc_checker.egg-info/dependency_links.txt 10 src/doc_checker.egg-info/requires.txt 6 -src/doc_checker.egg-info/top_level.txt 11 +src/doc_checker.egg-info/top_level.txt 12 +src/doc_checker/api/__init__.py 13 +src/doc_checker/api/dependencies.py 14 +src/doc_checker/api/health_router.py 15 +src/doc_checker/custom_types/__init__.py 16 +src/doc_checker/custom_types/config_types.py └ Updated uv.lock (+33 -23) 33 { url = "https://files.pythonhosted.org/packages/a9/16/7757fb84c819accc7d7e9af5291b261e4abb8627ee040f99e7d35ef2457a/anyio-4.16.0-py3-none-any.whl", hash = "sha256:c66ae634e 0b8228396b801741beb3294326d4b6102f7bba4041c0dfb58c29026", size = 134521, upload-time = "2026-11-01T11:20:05.59Z" }, 34] 35 36 -[[package]] 37 -name = "certifi" 38 -version = "2026.10.23" 39 -source = { registry = "https://pypi.org/simple" } 40 -sdist = { url = "https://files.pythonhosted.org/packages/fd/11/b524cbd9bbdaca58608e4cf4053fd463417549e1959fa8d3aa18319f9555/certifi-2026.10.23.tar.gz", hash = "sha256:a9895e6e2 -758b371e0545e321a3a8fd1fcfd9829ada8824a936dfb16aaaf36cd", size = 136225, upload-time = "2026-10-23T11:36:16.478Z" } 41 -wheels = [42 - { url = "https://files.pythonhosted.org/packages/32/d1/a3652fe2c19134d0d15922177821d863cbd7e842d53a21bbe7e8e3a1b5ac/certifi-2026.10.23-py3-none-any.whl", hash = "sha256:e9b -54de76e530bb6cf0ccd036787103acb58fd42ba930ea8bf948ddbaa2f61d1", size = 135082, upload-time = "2026-10-23T11:36:14.763Z" }, 43 -] 44 - 36 [[package]] 37 name = "click" 38 version = "8.5.1" ... 63 64 [package.dev-dependencies] 65 dev = [75 - { name = "httpx" }, 66 + { name = "httpx2" }, 67 { name = "pytest" }, 68 { name = "ruff" }, 69 { name = "ty" }, ... 78 79 [package.metadata.requires-dev] 80 dev = [90 - { name = "httpx", specifier = ">=0.28.1" }, 81 + { name = "httpx2", specifier = ">=2.16.0" }, 82 { name = "pytest", specifier = ">=9.2.0" }, 83 { name = "ruff", specifier = ">=0.17.4" }, 84 { name = "ty", specifier = ">=0.0.100" }, ... 111] 112 113 [[package]] 123 -name = "httpcore" 124 -version = "1.0.9" 114 +name = "httpcore2" 115 +version = "2.16.0" 116 source = { registry = "https://pypi.org/simple" } 117 dependencies = [… 62 more lines (ctrl+o to expand) └ (timeout 3m) └ Allowed by auto mode ● Update(src\doc_checker\main.py) └ Added 1 line, removed 1 line 21 22 def build_app() -> FastAPI: 23 """Uvicorn factory entrypoint: `uvicorn doc_checker.main:build_app --factory`.""" 24 - config = AppConfig() # ty: ignore[missing-argument] 24 + config = AppConfig() 25 configure_logging(APPLICATION_NAME, config.log_level) 26 logger.info(27 "application configured", ● Update(tests\unit\test_config.py) └ Added 1 line, removed 1 line 80 81 def test_config_is_immutable(app_config: AppConfig) -> None: 82 with pytest.raises(ValidationError): 83 - app_config.llm_model = "something-else" 83 + setattr(app_config, "llm_model", "something-else") # noqa: B010 - bypasses static read-only check on purpose ● Bash(uv run pytest -q 2>&1 | tail -2; uv run ruff check . && uv run ruff format --check . && uv run ty check 2>&1 | tail -3) └ ..... [100%] 20 passed in 0.35s All checks passed! … +2 lines (ctrl+o to expand) └ 2 files changed

(binary, mode only or too large to show) └ Allowed by auto mode ● All 20 tests, ruff, format and ty are green. I'm now checking whether a missing required variable makes Pydantic echo the API key into the error message, using a dummy key. · summary ● Bash(env -u LLM_MODEL -u LLM_FALLBACK_MODEL GEMINI_API_KEY=DUMMY-SECRET-123 LLM_FALLBACK_MODEL=x uv run python -c "from doc_checker.main import build_app; build_app(…) └ File "C:\projects\elta_hw\backend\.venv\Lib\site-packages\pydantic\main.py", line 280, in __init__ validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self) pydantic_core._pydantic_core.ValidationError: 1 validation error for AppConfig … +3 lines (ctrl+o to expand) └ Allowed by auto mode ● The startup error leaks the start of the real key into container logs, so I'm adding a load_config() that reports only field names and error types, written test-first. · summary ● Update(tests\unit\test_config.py) └ Added 1 line, removed 1 line 3 import pytest 4 from pydantic import ValidationError 5 6 - from doc_checker.config import AppConfig 6 +from doc_checker.config import AppConfig, ConfigurationError, load_config 7 from doc_checker.custom_types.config_types import LogLevel 8 9 ● Bash(cat >> tests/unit/test_config.py <<'EOF'…) └ ========== short test summary info ========== ERROR tests/unit/test_config.py !! Interrupted: 1 error during collection !! 1 error in 0.48s └ Updated tests\unit\test_config.py (+22 -0) 81 def test_config_is_immutable(app_config: AppConfig) -> None: 82 with pytest.raises(ValidationError): 83 setattr(app_config, "llm_model", "something-else") # noqa: B010 - bypasses static read-only check on purpose 84 + 85 + 86 +def test_load_config_returns_config(required_env: pytest.MonkeyPatch) -> None: 87 + assert load_config().llm_model == "test-primary-model" 88 + 89 + 90 +def test_load_config_error_names_fields_without_leaking_values(91 + required_env: pytest.MonkeyPatch, 92 +) -> None: 93 + required_env.delenv("LLM_MODEL") 94 + required_env.setenv("CHUNK_TOKEN_BUDGET", "0") 95 + 96 + with pytest.raises(ConfigurationError) as exc_info: 97 + load_config() 98 + 99 + message = str(exc_info.value) 100 + assert "LLM_MODEL" in message 101 + assert "CHUNK_TOKEN_BUDGET" in message 102 + assert "test-key-not-real" not in message 103 + assert "test-fallback-model" not in message 104 + assert exc_info.value.__cause__ is None 105 + assert exc_info.value.__suppress_context__ └ … 1 more file changed └ Allowed by auto mode ● Red (import error: ConfigurationError / load_config don't exist). Implementing: ● Update(src\doc_checker\config.py) └ Added 1 line, removed 1 line 1 from __future__ import annotations 2 3 -from pydantic import Field, SecretStr, field_validator 3 +from pydantic import Field, SecretStr, ValidationError, field_validator 4 from pydantic_settings import BaseSettings, SettingsConfigDict 5 6 from doc_checker.custom_types.config_types import LogLevel ● Bash(cat >> src/doc_checker/config.py <<'EOF'…) └ 8:from doc_checker.config import AppConfig, load_config 10:from doc_checker.logging_config import configure_logging 15:def create_app(config: AppConfig) -> FastAPI: … +11 lines (ctrl+o to expand) └ Updated src\doc_checker\config.py (+19 -0) 38 if not value.get_secret_value().strip(): 39 raise ValueError("GEMINI_API_KEY must not be blank") 40 return value 41 + 42 + 43 +class ConfigurationError(Exception): 44 + """The environment does not describe a valid configuration.""" 45 + 46 + 47 +def load_config() -> AppConfig: 48 + """Build the config, reporting problems by variable name only. 49 + 50 + Pydantic's own error text echoes input values, which would leak GEMINI_API_KEY into logs. 51 + """ 52 + try: 53 + return AppConfig() 54 + except ValidationError as exc: 55 + problems = [56 + f"{'.'.join(str(part) for part in error['loc']).upper()} ({error['type']})" 57 + for error in exc.errors(include_input=False) 58 +] 59 + raise ConfigurationError(f"Invalid configuration: {', '.join(problems)}") from None └ Updated src\doc_checker\main.py (+2 -2) 5 from fastapi import FastAPI 6 7 from doc_checker.api.health_router import router as health_router 8 -from doc_checker.config import AppConfig 8 +from doc_checker.config import AppConfig, load_config 9 from doc_checker.constants import API_PREFIX, APP_TITLE, APPLICATION_NAME 10 from doc_checker.logging_config import configure_logging 11 ... 21 22 def build_app() -> FastAPI: 23 """Uvicorn factory entrypoint: `uvicorn doc_checker.main:build_app --factory`.""" 24 - config = AppConfig() 24 + config = load_config() 25 configure_logging(APPLICATION_NAME, config.log_level) 26 logger.info(27 "application configured", └ … 2 more files changed └ Allowed by auto mode ● 22 passing. Now a real end-to-end smoke test: re-check the leak, then boot uvicorn with dummy values and hit /api/health. ● Bash(echo "--- missing var:"; env -u LLM_MODEL GEMINI_API_KEY=DUMMY-SECRET-123 LLM_FALLBACK_MODEL=x uv run python -c "from doc_checker.main import build_app; build_a…) └ --- missing var: doc_checker.config.ConfigurationError: Invalid configuration: LLM_MODEL (missing) --- server: … +16 lines (ctrl+o to expand) └ (timeout 1m 30s) └ Allowed by auto mode ● The server boots and its health check and structured JSON logs work, and the startup error no longer leaks anything. I'm adding ignore rules for the backend artifacts and aligning .env.example with the plan next. · summary ● Bash(cd /c/projects/elta_hw && cat >> .gitignore <<'EOF'…) └ ?? .env.example ?? .gitignore ?? backend/.python-version … +24 lines (ctrl+o to expand) └ Updated .env.example (+13 -2) 1 -GEMINI_API_KEY= 2 -GEMINI_MODEL=gemini-3.8-flash 1 +# Required 2 +GEMINI_API_KEY= 3

+LLM_MODEL=gemini-3.8-flash 4
+LLM_FALLBACK_MODEL=gemini-2.5-flash 5 +
6 +# Optional (values shown
are the defaults) 7 +# LLM_TEMPERATURE=0
8 +# LLM_MAX_CONCURRENCY=2
9 +# CHUNK_TOKEN_BUDGET=30000
10 +# CHUNK_OVERLAP_BLOCKS=2
11 +# MAX_UPLOAD_MB=20
12 +# DATABASE_URL=sqlite:///./doc_checker.db
13 +# LOG_LEVEL=INFO
└ Updated .gitignore (+8 -1) 6 .env 7 .claude 8 CLAUDE.md 9 -apm*
9 +apm*
10 +# Python backend
11 +.venv/
12 +__pycache__/
13 +*.pyc
14 +.pytest_cache/ 15 +.ruff_cache/
16 +*.db
└ Allowed by auto mode ● Bash(echo "*.egg-info/" >> .gitignore
&& git status --short --untracked-files=all | grep -c egg-info) └ 0 └ Updated .gitignore (+1 -0) 14 .pytest_cache/ 15 .ruff_cache/
16 *.db 17 +*.egg-info/ ● Finally,
recording the scaffold decisions in the plan so it stays the source of truth: ● Update(knowledge\plans\01-solution-plan.md) └ Added 1 line,
removed 1 line 61 62 ### Q1. Stack 63 64 -- Backend: Python 3.12, **FastAPI**, Pydantic v2, SQLModel, pydantic-settings,
`pytest`. Packaging: **`pyproject.toml` managed with `uv`** (`uv add`, `uv sync`, committed `uv.l -ock`), linting with `ruff`, type checking
with `ty`. No `requirements.txt`. 64 +- Backend: Python 3.13 (3.14 is blocked
by Application Control on the dev machine), **FastAPI**, Pydantic v2, SQLModel, pydantic-settings, `pytest`. Packaging: **`pyproject.tom
+l` managed with `uv`** (`uv add`, `uv sync`, committed `uv.lock`), linting with `ruff`, type checking with `ty`. No `requirements.txt`.
65 - Frontend: **Vite + React + TypeScript**, Tailwind, TanStack Query, React Hook Form + Zod, React Router, Vitest. 66 -
Infra: `docker-compose.yml` with `backend` and `frontend` services. The frontend is built and served by nginx, which reverse-proxies `/api` to
the backend (single origin, no CORS setup). 67 ● Update(knowledge\plans\01-solution-plan.md) 441 +Config is read from
process env only, no implicit `.env` lookup. Locally: `cd backend && uv run --env-file ../.env uvicorn doc_checker.main:build_app --factory --
reload`. In D +ocker: Compose `env_file: .env`.
442 +
443 | Variable |
Required | Example / default | Purpose | 444 |---|---|---|---| 445 | `GEMINI_API_KEY` | yes | `AIza...` (your key) | Google AI Studio key | ●
Update(knowledge\plans\01-solution-plan.md) └ Added 2 lines 473 474 If a skill conflicts with this plan, the skill wins and this plan is
updated. 475 476 +Layout decision: `container-architecture` targets batch jobs (`JOB_NAME` dispatch, exit codes). The backend is a
long-running HTTP API, so it uses the `python-layout` `src/doc +_checker/` layout, borrowing only the frozen fail-fast config and wire-
once-at-startup ideas (`create_app(config)` + `app.state`). 477 +
478 ### TDD cycle (every feature and bug fix) 479 480 1.
Interface: define the protocol / function signature and Pydantic models, no logic. ● The backend scaffold is done and runs: 22 tests pass, and
ruff, formatting and ty are all clean. I started the server with dummy values and /api/health returned the configured models. Nothing is
committed. What's in backend/ - Setup: pyproject.toml managed with uv, a committed uv.lock, and Python pinned to 3.13 in .python-version.
- config.py: - Settings are read from environment variables once and can't be changed afterwards. - The app refuses to start if
GEMINI_API_KEY, LLM_MODEL or LLM_FALLBACK_MODEL is missing or blank, or if any value is invalid. - The API key is masked in any
printout of the settings. - logging_config.py: every log line is one JSON object with the fields from the Python skill. - main.py:
create_app(config) builds the app (tests call it with a test config). The server is started with uvicorn doc_checker.main:build_app --factory. -
Health endpoint: GET /api/health returns the status and both model names, never the key. A leak I found and fixed. When a required variable
was missing, Pydantic's error message printed the start of GEMINI_API_KEY; my test with a dummy key showed 'DUMMY.... I added
load_config(), which reports only the variable name and the problem, for example LLM_MODEL (missing). A test now covers this. Changes
from the plan (the plan file is updated): - Python 3.13 instead of 3.12: an Application Control policy blocks the Python 3.14 install on this
machine, so I used the working 3.13. - Folder layout: the container skill's layout is built for batch jobs that run once and exit. The backend is a
server that keeps running, so I used the Python skill's src/ layout. - Default values: they live in config.py instead of constants.py, because the
Python layout skill says those two files never import each other. - httpx2 instead of httpx for tests: the test client warned that httpx is deprecated
with it. - .gitignore: I added Python build and cache folders at the end, outside the managed block. - .env.example: updated to the plan's
variable names. - TDD skill: it says to hand TDD work to the senior-python-developer subagent. I did this small piece myself, still writing failing
tests before the code. I can delegate the larger features. You need to edit your .env. It still has GEMINI_MODEL. Change it to: dotenv
LLM_MODEL=gemini-3.8-flash LLM_FALLBACK_MODEL=gemini-2.5-flash To run the server locally: cd backend && uv run --env-file ../.env
uvicorn doc_checker.main:build_app --factory --reload The next step in the plan is turning DOCX and TXT files into numbered blocks, built test-
first. Should I continue with that? ✻ Baked for 4m 25s · done 7:47 PM ❯ before the next step, I want to handle the command for running the
application (with docker), tests, ruff check and lint check using poe (uv run poe ...), add those to the pyproject.toml and keep in mind you
should add relevant command as the development continues. ●
Skill(container-deployment) └ Successfully loaded skill ● There's no Dockerfile yet, so I'll add a backend Dockerfile and a root docker-
compose.yml with a multi-stage build, a non-root user, and a .dockerignore. · summary ● Write(backend\Dockerfile) └ Wrote 30 lines to
backend\Dockerfile 1 # syntax=docker/dockerfile:1 2 FROM --platform=linux/amd64 python:3.13-slim AS builder 3 4 COPY --
from=ghcr.io/astral-sh/uv:0.11.21 /uv /usr/local/bin/uv 5 ENV UV_COMPILE_BYTECODE=1 \ 6 UV_LINK_MODE=copy \ 7
UV_PYTHON_DOWNLOADS=never 8 9 WORKDIR /app … +21 lines (ctrl+o to expand) ● Write(backend\.dockerignore) └ Wrote
13 lines to backend\.dockerignore 1 .git/ 2 tests/ 3 .env 4 .env.* 5 .venv/ 6 __pycache__/ 7 *.pyc 8
.pytest_cache/ 9 .ruff_cache/ 10 *.egg-info/ … +3 lines (ctrl+o to expand) ● Write(docker-compose.yml) └ Wrote 19 lines to docker-
compose.yml 1 services: 2 backend: 3 build: ./backend 4 env_file: .env 5 environment: 6
DATABASE_URL: sqlite:////data/doc_checker.db 7 volumes: 8 - backend-data:/data 9 ports: 10 - "8000:8000"
… +9 lines (ctrl+o to expand) ● Bash(cd /c/projects/elta_hw/backend && uv add --dev poethepoet -q 2>&1 | tail -2 && cat >> pyproject.toml
<<'EOF'…) └ --no-ansi Force disable ANSI output Configured tasks: … +22 lines (ctrl+o to expand) └ Updated
pyproject.toml (+14 -0) 30 [dependency-groups] 31 dev = [32 "httpx2>=2.16.0", 33 + "poethepoet>=0.49.0",
34 "pytest>=9.2.0", 35 "ruff>=0.17.4", 36
"ty>=0.0.100", 37] 38 +
39 +[tool.poe.tasks] 40 +dev = { cmd
= "uvicorn doc_checker.main:build_app --factory --reload --port 8000", envfile = "../.env", help = "Run the API locally with hot reload (reads
../.env)" } 41 +test = { cmd = "pytest", help = "Run unit tests (extra args are passed to pytest)" }
42 +lint = { cmd = "ruff check .", help = "Lint with ruff" }
43 +lint-fix = { cmd = "ruff check . --fix", help = "Lint and auto-fix safe violations" }
44 +format = { cmd = "ruff format .", help = "Format code with ruff" }
45 +format-check = { cmd = "ruff format --check .", help = "Verify formatting without writing" }

46 +typecheck = { cmd = "ty check", help = "Type check with ty" }
47 +check = { sequence = ["lint", "format-check", "typecheck", "test"], help = "Full quality gate: lint, format check, type check, tests" }
48 +up = { cmd = "docker compose up --build -d", cwd = "..", help = "Build and start the stack in Docker (detached)" }
49 +down = { cmd = "docker compose down", cwd = "..", help = "Stop the Docker stack" }
50 +logs = { cmd = "docker compose logs -f", cwd = "..", help = "Follow Docker stack logs" }
└ Updated uv.lock (+14 -0) 64 [package.dev-dependencies] 65 dev = [66 { name = "httpx2" }, 67 + { name = "poethepoet" }, 68 { name = "pytest" }, 69 { name = "ruff" }, 70 { name = "ty" }, … 80 [package.metadata.requires-dev] 81 dev = [82 { name = "httpx2", specifier = ">=2.16.0" }, 83 + { name = "poethepoet", specifier = ">=0.49.0" }, 84 { name = "pytest", specifier = ">=9.2.0" }, 85 { name = "ruff", specifier = ">=0.17.4" }, 86 { name = "ty", specifier = ">=0.0.100" }, … 271 { url = "https://files.pythonhosted.org/packages/40/9e/2b38731e0fc536806f16490e1a12d7f0dc2a1235aa8cc07bcc75416a7daa/pluggy-1.7.0-py3-none-any.whl", hash = "sha256:7dd7b0d8 832ba3cb632c306926ded123429211b83641b35dc5c41ad2d34f9bec", size = 27082, upload-time = "2026-10-15T09:50:56.808Z" }, 272] 273 274 +[[package]]
275 +name = "poethepoet"
276 +version = "0.49.0"
277 +source = { registry = "https://pypi.org/simple" } 278 +dependencies = [279 + { name = "pyyaml" }, 280 +]
281 +sdist = { url = "https://files.pythonhosted.org/packages/4e/7e/160d391368d359f7dd6dd647575499c395687beb4a6b2ebfbfbf8a20b817/poethepoet-0.49.0.tar.gz", hash = "sha256:256b72c05 +7b5f60341bbe09498212939e558341fac4ab59d0ce478aca04f0d2a", size = 157750, upload-time = "2026-10-15T06:08:57.094Z" }
282 +wheels = [
283 + { url = "https://files.pythonhosted.org/packages/a1/f5/a2404b9cb3c299cd148533d07e1a378b60aa4ae640c69fb522596ca157ae/poethepoet-0.49.0-py3-none-any.whl", hash = "sha256:97b +456f3c52c9af75d4bc504e0f5cfee44545a84f446ba645a0a8f6a0131422f", size = 195985, upload-time = "2026-10-15T06:08:55.332Z" }, 284 +]
285 +
286 [[package]] 287 name = "pydantic" 288 version = "2.14.3" └ (timeout 3m) └ Allowed by auto mode ● The quality gate passes through poe. Now verifying Docker: I'll build the image and run it with dummy values, because your .env doesn't have the new variable names yet. ● Bash(cd /c/projects/elta_hw && docker compose config -q && echo "compose file valid" && docker build -q -t doc-checker-backend:smoke ./backend 2>&1 | tail -3 && dock…) └ compose file valid
sha256:5405a00d45f51bd5bfc11c2762e518a3c35894b7708ed4fa15d04a8448c28d1f … +4 lines (ctrl+o to expand) └ (timeout 10m) └ Allowed by auto mode ● The image builds and runs as the non-root user, failing cleanly when variables are missing, but the health check came back empty, likely because curl hit the port before uvicorn was listening. I'm checking that now. · summary ● Bash(docker run -d --rm --name dc-smoke -p 8766:8000 -e GEMINI_API_KEY=DUMMY -e LLM_MODEL=gemini-3.8-flash -e LLM_FALLBACK_MODEL=gemini-2.5-flash doc-checker-backend…) └ { "status": "ok", "llm_model": "gemini-3.8-flash", … +14 lines (ctrl+o to expand) └ Allowed by auto mode ● It works: the container serves /api/health and logs in JSON. Saving your "keep poe tasks up to date" instruction to memory and adding it to the plan: Wrote 1 memory (ctrl+o to expand) ● Bash(cd /c/Users/shilo/.claude/projects/C--projects-elta-hw/memory && echo "- [Poe task runner](poe-task-runner.md) — expose every command as a poe task; keep adding…) └ - [elta_hw exam project](elta-hw-exam-project.md) — 4h take-home exam, Gemini free tier, plan in knowledge/plans/01-solution-plan.md - [Poe task runner](poe-task-runner.md) — expose every command as a poe task; keep adding them └ Shell cwd was reset to C:\projects\elta_hw └ Allowed by auto mode ● Update(knowledge\plans\01-solution-plan.md) └ Added 14 lines 475 476 Layout decision: `container-architecture` targets batch jobs (`JOB_NAME` dispatch, exit codes). The backend is a long-running HTTP API, so it uses the `python-layout` `src/doc _checker/` layout, borrowing only the frozen fail-fast config and wire-once-at-startup ideas (`create_app(config)` + `app.state`). 477 478 +### Commands (poe) 479 +
480 +Every command is a poe task in `backend/pyproject.toml`, run from `backend/` as `uv run poe <task>`. New runnable things get a task in the same change.
481 +
482 +| Task | Does |
483 +|---|---|
484 +| `dev` | Local API with hot reload, reads `../.env` |
485 +| `test` | pytest (extra args pass through, e.g. `uv run poe test -k config`) |
486 +| `lint` / `lint-fix` | `ruff check` (lint) |
487 +| `format` / `format-check` | `ruff format` |
488 +| `typecheck` | `ty check` |
489 +| `check` | lint + format-check + typecheck + test (the quality gate) |
490 +| `up` / `down` / `logs` | Docker Compose stack at the repo root |
491 +
492 ### TDD cycle (every feature and bug fix) 493 494 1. **Interface:** define the protocol / function signature and Pydantic models, no logic. ● All commands now run through poe from backend/ as uv run poe <task>. uv run poe check passes (22 tests). I also built and ran the Docker image with dummy values: /api/health responded and the container runs as a non-root user. I haven't run poe up itself, because your .env doesn't have the new variable names yet.

Task	Does
dev	Runs the API locally with hot reload, reading ../.env
test	Runs the tests; extra arguments go to pytest, e.g. uv run poe test -k config
lint / lint-fix	ruff check (this is the lint check)
format / format-check	ruff format; the check variant only verifies
typecheck	ty check
check	lint, format check, type check and tests in one go
up / down / logs	Starts, stops and follows the Docker stack

Docker files, since up needed something to run: - backend/Dockerfile: follows the container skill. It builds in two stages and the final image runs as a non-root user. Dependencies are installed before the source code is copied, so rebuilds after code changes are fast. The image is 234 MB. - backend/.dockerignore: keeps .env, tests, .venv and caches out of the build. - docker-compose.yml at the repo root: has only the

backend for now. It reads .env, keeps the SQLite database in a named volume, exposes port 8000 and has a health check. The frontend service gets added when we build it. Not proven yet: poe up itself. With your current .env (GEMINI_MODEL instead of LLM_MODEL / LLM_FALLBACK_MODEL), the container will stop at startup and keep restarting with Invalid configuration: LLM_MODEL (missing), LLM_FALLBACK_MODEL (missing). Once you update .env, uv run poe up should work. I saved your instruction to memory and added it to the plan: whenever something new becomes runnable (integration tests, frontend build and tests, seeding, skill scripts), it gets a poe task in the same change. The next step is turning DOCX and TXT files into numbered blocks, built test-first. Should I continue? /////////// should I run any test currently?

Response: No, you do not need to manually run any tests right now. Claude just confirmed that all 22 tests, `ruff`, `ty`, and the `poe check` quality gate are completely green, and it has already successfully smoke-tested the Docker image startup.

The only thing you need to do before Claude proceeds is update your `.env` file to match the new variable names the system is looking for (which caused the startup to fail with dummy values).

Open your `.env` file and make sure it looks exactly like this:

```
GEMINI_API_KEY=your_actual_api_key_string_here
LLM_MODEL=gemini-3.8-flash
LLM_FALLBACK_MODEL=gemini-2.5-flash
```

Once you have updated the `.env` file, tell Claude to proceed to the next step (Extraction).

User prompt: how to set up a api key for a model using open router

Response: Here is how to get an OpenRouter API key and set it up for your project.

Note: You mentioned earlier that you do not want to pay for model calls. While OpenRouter does offer a subset of "free" open-source models (like Llama 3 8B), accessing top-tier models (like Claude 3.5 Sonnet or GPT-4o) requires adding prepaid credits ($5 minimum) with a credit card.

Step 1: Generate the Key

1. Go to **openrouter.ai** and sign in.
2. If you want to use paid models, click on the **Credits** tab to add a balance.
3. Click on the **Keys** tab in the top navigation menu.
4. Click **Create Key**. Give it a name (e.g., `doc-checker-dev`) and copy the generated string (it will start with `sk-or-v1-`).

Step 2: Update Your `.env` File

If you decide to pivot your plan to use OpenRouter, you will need to swap out your Gemini configuration.

```
OPENROUTER_API_KEY=sk-or-v1-your-actual-api-key
LLM_MODEL=anthropic/claude-3.5-sonnet  # OpenRouter uses provider/model format
```

Step 3: Call it in Python

OpenRouter uses an OpenAI-compatible API. You do not install an "openrouter" SDK; instead, you use the official `openai` Python SDK and simply point the base URL to OpenRouter.

If you are updating your project, you would run `uv add openai` and write your client like this:

```
import os
from openai import OpenAI

client = OpenAI(
  base_url="https://openrouter.ai/api/v1",
  api_key=os.getenv("OPENROUTER_API_KEY"),
)

response = client.chat.completions.create(
  model=os.getenv("LLM_MODEL", "anthropic/claude-3.5-sonnet"),
  messages=[
    {"role": "system", "content": "You are a document compliance reviewer."},
    {"role": "user", "content": "Check this text..."}
  ],
  # OpenRouter supports strict JSON output for compatible models
  response_format={"type": "json_object"}
)

print(response.choices[0].message.content)
```

If you are sticking to your finalized plan with Gemini to keep things 100% free: You can ignore this setup, ensure your `.env` has the correct `GEMINI_API_KEY` and `LLM_MODEL=gemini-3.8-flash`, and tell Claude to proceed to the next step (Extraction).

User prompt: I want free models only with big context

Response: If you strictly want **100% free models with a massive context window**, you must completely abandon the OpenRouter idea and stick with your finalized **Gemini API (Google AI Studio)** plan.

Here is why Gemini is the only provider that fits your requirements for this exam:

- **The Context Window:** Google AI Studio's free tier gives you a 1-million to 2-million token context window. This is what allows your pipeline to pass massive document chunks directly to the model.
- **The Free OpenRouter Problem:** While OpenRouter does offer a few free models (like Llama 3 8B or Mistral), their context windows are usually capped at 8k to 32k tokens. They will choke on your DOCX chunks, forcing you to rewrite your architecture to use complex vector databases (RAG) which you absolutely do not have time for in a 4-hour exam.

Your Immediate Next Steps:

1. Ignore the OpenRouter setup.
2. Ensure your `.env` file looks exactly like this to match what Claude just built:

```
GEMINI_API_KEY=your_actual_api_key_from_google_ai_studio
LLM_MODEL=gemini-3.8-flash
LLM_FALLBACK_MODEL=gemini-2.5-flash
```

3. Go back to Claude Code and tell it: **"The `.env` is updated with the Gemini keys. Proceed to the next step: turning DOCX and TXT files into numbered blocks (Extraction), built test-first."**

User prompt: but we have this problme with gemini: ● Both configured model names exist for your key, but the generation call returned HTTP 503 UNAVAILABLE from gemini-3.8-flash even after the SDK's retries. I'm checking Google's error message to tell a temporary overload from an account problem before changing anything. · summary ● Bash(uv run --env-file ../.env python - 2>&1 <<'EOF' | grep -v "SyntaxWarning\|description=\|Output only" import asyncio, os…) └ gemini-3.8-flash -> HTTP 503 UNAVAILABLE | This model is currently experiencing high demand. Spikes in demand are usually temporary. Please try again later. gemini-2.5-flash -> HTTP 404 NOT_FOUND | This model models/gemini-2.5-flash is no longer available to new users. Please update your code to use models/gemini-3.8-f lash for the latest features and improvements. We recommend you to use the Interactions API (https://aistudio.google.com/docs/get-started). └ (timeout 2m) └ Allowed by auto mode ● gemini-3.8-flash is only temporarily overloaded (503), so the problem is not in our code, while gemini-2.5-flash returns 404 as a fallback and my availability test missed it because it only checked models.list. I'm now testing which other models your key can really call. · summary ● Bash(uv run --env-file ../.env python - 2>&1 <<'EOF' | grep -v "SyntaxWarning\|description=\|Output only" import asyncio, os…)

Response: You just hit the classic tradeoff of the free tier: capacity drops during peak hours (the 503), and Google aggressively deprecates older models (the 404 on 2.5-flash).

Let Claude's script finish running. It is doing exactly the right thing by querying the API to see which models your specific key actually has access to right now, bypassing the outdated documentation.

Once the script outputs the list of working models, here is how you handle it:

1. **Update your Fallback:** Change `LLM_FALLBACK_MODEL` in your `.env` to whatever free lightweight model the script finds (most likely `gemini-3.5-flash-lite` or a `gemini-3.8-flash-lite` variant).
2. **Handle the 503 (Temporary Overload):** Since `gemini-3.8-flash` is just experiencing a temporary traffic spike, you don't need to abandon it permanently. However, you need to keep moving. Temporarily set your main `LLM_MODEL` to the working fallback model just to get your tests passing and the Extraction pipeline built. You can swap it back to 3.8-flash for the final end-to-end run later tonight.

Tell Claude: **"Update the `.env` file to use the working flash/lite model you just found as the temporary `LLM_MODEL`, and proceed with building the DOCX/TXT Extraction step test-first."**

User prompt: which are frer tiers?

Response: Based on the official Google AI Studio documentation on your screen and the error messages Claude received, here are the exact free-tier models you should use for your `.env` file right now:

1. **Primary Model:** `gemini-3.8-flash`
   - *Status:* This is the flagship free-tier model. The 503 error Claude got just means it's temporarily busy (high traffic spike). You should keep this as your main model.
2. **Fallback Model:** `gemini-3.5-flash-lite`
   - *Status:* The documentation explicitly states that `gemini-2.5-flash` is deprecated and all traffic is being routed to `gemini-3.5-flash-lite`. This is your new, ultra-fast free-tier fallback model.

Your Immediate Action: Update your `.env` file to look exactly like this:

```
GEMINI_API_KEY=your_actual_api_key_here
LLM_MODEL=gemini-3.8-flash
LLM_FALLBACK_MODEL=gemini-3.5-flash-lite
```

Once you've saved that, tell Claude: **"The `.env` is updated with the correct Gemini 3.8 and 3.5 Flash models. Proceed to the next step: turning DOCX and TXT files into numbered blocks (Extraction), built test-first."**

User prompt: ❯ (base) C:\projects\elta_hw\backend [main ≡ +6 ~0 -0 !]> uv run poe test-integration
Poe => pytest -m integration -s
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\projects\elta_hw\backend
configfile: pyproject.toml testpaths: tests
plugins: anyio-4.16.0
collected 127 items / 124 deselected / 3 selected

tests\integration\test_gemini_live.py .. model=gemini-3.5-flash quotes=['maximum detection range is 50 km'] dropped=0 .

======================= 3 passed, 124 deselected in 26.54s ========================
(base) C:\projects\elta_hw\backend [main ≡ +6 ~0 -0 !]>
● All 3 integration tests passed. That's the first real end-to-end result. Marking the LLM client criteria as done in the plan: ● Bash(cd /c/projects/elta_hw && python - <<'EOF' p = "knowledge/plans/01-solution-plan.md"…) └ done └ Updated knowledge\plans\01-solution-plan.md (+2 -2) 323 - [x] Duplicate from overlapping chunks appears once (and a span inside a longer span of the same rule is merged). 324 325 LLM client: 326 -- [ ] 429 on the primary model triggers one retry, then the fallback model. 327 -- [ ] Safety-blocked response raises a typed error that the service records as a failed chunk. 326 +- [x] Capacity errors on the primary (429 quota, 503 overload, timeout) switch to the fallback model; 5xx is retried by the SDK first; other errors fail wit +hout fallback. 327 +- [x] Safety-blocked response raises a typed error (`LlmBlockedError`) that the service records as a failed chunk. 328 329 Service and persistence: 330 - [ ] Only active rules (or the selected subset) are sent to the model. └ Allowed by auto mode ● The Gemini client step is done, and your run confirms it works against the real API: - Both models in your .env answered (gemini-3.5-flash and gemini-3.5-flash-lite). - The exam example works end to end. The pipeline went through extraction, chunking, the Gemini call and quote verification, and returned exactly one violation: "maximum detection range is 50 km". - No false alarms: "version 2.1" and "March 2024" were not flagged. - No invented quotes: the quote check dropped 0 quotes, so the model quoted the document exactly. - Speed: 26.5 s for all 3 tests. I've marked the LLM client success criteria as done in the plan. Nothing is committed yet. Next is the check service: it runs the whole pipeline over every chunk, sends a few chunks to Gemini at a time (2 by default, LLM_MAX_CONCURRENCY), and produces one combined result. If some chunks fail, the result is marked "partial" with the failed chunks listed, never shown as clean. After that come the database and API step, and then you can upload a real DOCX to the Docker container. Should I continue? ✲ Sautéed for 10s · done 8:30 PM ❯ yes, continue with the check service step. keep in mind to not actaully run it with the model at least untill we reach the final tests because it takes too much time and wastes model calls
● Understood: no live model calls until the final tests. Everything in this step uses a fake LLM client. Saving that rule so it holds in later sessions too: Wrote 1 memory (ctrl+o to expand) ● Bash(echo "- [No live LLM calls](no-live-llm-calls.md) — no real Gemini calls until the final tests; use the fake LlmClient" >> /c/Users/shilo/.claude/projects/C--pr…) └ __init__.py └ Created src\doc_checker\services__init__.py (+0 -0) └ Created tests\unit\services__init__.py (+0 -0) └ Allowed by auto mode ● Interface first: the result types and the service signature. ● Write(src\doc_checker\custom_types\check_types.py) └ Wrote 50 lines to src\doc_checker\custom_types\check_types.py 1 from __future__ import annotations 2 3 from dataclasses import dataclass 4 from enum import StrEnum 5 6 from doc_checker.custom_types.document_types import Block, DocumentFormat 7 from doc_checker.custom_types.rule_types import CheckRule 8 from doc_checker.custom_types.violation_types import VerifiedViolation 9 9 10 class CheckError(Exception): … +23 lines (ctrl+o to expand) ● Failing tests. The fake LLM decides its answer from the chunk's blocks and records how many calls are running at once, so the concurrency limit is actually measured: ● Write(tests\unit\services\test_compliance_checker.py) └ Wrote 231 lines to tests\unit\services\test_compliance_checker.py 1 from __future__ import annotations 2 3 import asyncio 4 from collections.abc import Callable, Sequence 5 from dataclasses import dataclass, field 6 7 import pytest 8 9 from doc_checker.custom_types.check_types import ChunkFailureReason, CheckStatus 10 from doc_checker.custom_types.chunk_types import Chunk … +221 lines (ctrl+o to expand) ● Bash(uv run poe test -q tests/unit/services 2>&1 | tail -2) └ FAILED tests/unit/services/test_compliance_checker.py::test_models_used_lists_every_model_that_answered - NotImplementedError 16 failed, 1 passed in 0.28s └ Allowed by auto mode ● 16 tests fail on NotImplementedError, and one passes falsely because NotImplementedError subclasses RuntimeError; I'm implementing the code now. · summary ● Write(src\doc_checker\services\compliance_checker.py) └ Added 141 lines, removed 4 lines 1 from __future__ import annotations 2 3 +import asyncio
4 +import logging
5 +import time 6 from collections.abc import Sequence 7 +from dataclasses import dataclass
8 5 -from doc_checker.custom_types.check_types import CheckResult
9 +from doc_checker.chunking.block_chunker import build_chunks
10 +from doc_checker.custom_types.check_types import (11 + CheckResult, 12 + CheckStatus,
13 + ChunkFailureReason,
14 + FailedChunk,
15 +) 16 +from doc_checker.custom_types.chunk_types import Chunk 17 +from doc_checker.custom_types.document_types import Block, ExtractedDocument 18 +from doc_checker.custom_types.llm_types import LlmCheckOutcome 19 from doc_checker.custom_types.rule_types import CheckRule 20 +from doc_checker.custom_types.violation_types import VerifiedViolation
21 +from doc_checker.extraction.document_extraction import extract_document
22 from doc_checker.llm.llm_client import LlmClient 23 +from doc_checker.llm.llm_errors import (24 + LlmBlockedError,
25 + LlmError,
26 + LlmOverloadedError, 27 + LlmQuotaExceededError, 28 + LlmResponseError,
29 + LlmTimeoutError,
30 +)
31 +from doc_checker.verification.quote_verifier import verify_violations
32 33 +logger = logging.getLogger(__name__) 34 35 +# Subclasses before their base (LlmOverloadedError and LlmTimeoutError extend LlmUnavailableError).
36 +_FAILURE_REASONS: tuple[tuple[type[LlmError], ChunkFailureReason], ...] = (

```
37 +    (LlmQuotaExceededError, ChunkFailureReason.QUOTA_EXCEEDED),
38 +    (LlmOverloadedError, ChunkFailureReason.OVERLOADED),                                                    39
+    (LlmTimeoutError, ChunkFailureReason.TIMEOUT),                                                                  40 +
(LlmBlockedError, ChunkFailureReason.BLOCKED),                                                                  41 +
(LlmResponseError, ChunkFailureReason.INVALID_RESPONSE),                                                                42 +)
                                                                                                                43 +
                                                                                                        44 +
                                                                45  class CheckError(Exception):      46      """Base for check-level failures that are not extraction or LLM errors."""      47
...      50      """A check was requested without any rule to check against."""      51      52      53 +@dataclass(frozen=True)
                                                                                                                54 +class _ChunkOutcome:
                                                                                                        55 +    chunk: Chunk
                                                                                                56 +    outcome: LlmCheckOutcome | None
57 +    failure: FailedChunk | None                                                                                58 +
                                                                                                        59 +
                                                                60  class ComplianceChecker:      61      def __init__(      62          self, llm: LlmClient,
token_budget: int, overlap_blocks: int, max_concurrency: int      63      ) -> None:      22 -          raise NotImplementedError
                                                                                                        64 +          self._llm = llm
                                                                                                65 +          self._token_budget = token_budget
66 +          self._overlap_blocks = overlap_blocks                                                                67 +
self._max_concurrency = max_concurrency                                                                         68          24 -      async def
check(self, filename: str, content: bytes, rules: Sequence[CheckRule]) -> CheckResult:                          69 +      async def
check(                                                                                                  70 +          self, filename: str, content: bytes,
rules: Sequence[CheckRule]                                                                              71 +      ) -> CheckResult:
                                                                                                72          """Extract, chunk, ask the model per chunk (bounded concurrency), verify, and
merge.      73          74          A chunk whose LLM call fails is recorded in `failed_chunks` (status PARTIAL, or FAILED      75          when no
chunk succeeded); it is never reported as clean. Raises NoRulesSelectedError,      76          and lets ExtractionError subtypes and unexpected
exceptions propagate.      77          """          31 -          raise NotImplementedError
                                                                                        78 +          if not rules:                                                                   79 +              raise
NoRulesSelectedError("Select at least one rule to check the document against.")                                 80 +
                                                                                                        81 +          started = time.monotonic()
                                                                                                82 +          document = extract_document(filename, content)
                                                                                        83 +          chunks = build_chunks(document.blocks, self._token_budget, self._overlap_blocks)
                                                                                84 +          outcomes = await self._check_chunks(rules, chunks)
                                                                                85 +                                                                                                  86 +
succeeded = [item.outcome for item in outcomes if item.outcome is not None]                                     87 +          failed
= tuple(item.failure for item in outcomes if item.failure is not None)                                  88 +          candidates =
[candidate for outcome in succeeded for candidate in outcome.candidates]                                        89 +          verification =
verify_violations(candidates, document, {rule.rule_id for rule in rules})                               90 +
                                                                                                91 +          result = CheckResult(
                                                                                        92 +              document_name=filename,
                                                                                93 +              document_format=document.document_format,
                                                                        94 +              status=_status(chunk_count=len(chunks), failed_count=len(failed)),                                  95
+              rules=tuple(rules),                                                                               96 +
violations=verification.violations,                                                                      97 +
context_blocks=_context_blocks(document, verification.violations),                                              98 +
block_count=len(document.blocks),                                                                       99 +
chunks_total=len(chunks),                                                                               100 +              failed_chunks=failed,
                                                                                                101 +              dropped_count=len(verification.dropped),
                                                                                        102 +              models_used=tuple(sorted({outcome.model_used for outcome in
succeeded})),                                                                                   103 +              duration_ms=round((time.monotonic() - started) * 1000),
                                                                                        104 +          )
                                                                                105 +          logger.info(                                                                    106 +              "check
finished",                                                                                              107 +              extra={
                                                                                        108 +                  "status": result.status,
                                                                                109 +                  "rule_count": len(rules),
    110 +                  "chunk_count": result.chunks_total,                                                      111 +
"failed_chunk_count": len(failed),                                                                      112 +                  "violation_count":
len(result.violations),                                                                         113 +                  "dropped_count": result.dropped_count,
                                                                                        114 +                  "duration_ms": result.duration_ms,
                                                                                115 +              },
        116 +          )                                                                                       117 +          return result
                                                                                                        118 +
                                                                                        119 +      async def _check_chunks(
                                                                                120 +          self, rules: Sequence[CheckRule], chunks: Sequence[Chunk]
                                                                                121 +      ) -> list[_ChunkOutcome]:                                                                   122 +
semaphore = asyncio.Semaphore(self._max_concurrency)                                                            123 +
                                                                                        124 +          async def check_one(chunk: Chunk) ->
_ChunkOutcome:                                                                                  125 +              async with semaphore:
                                                                                        126 +                  try:
                                                                                127 +                      outcome = await self._llm.find_violations(rules, chunk)
    128 +                  except LlmError as exc:                                                                  129 +
logger.warning(                                                                                 130 +                      "chunk check failed",
                                                                                131 +                      extra={"chunk_id": chunk.chunk_id, "error_type":
type(exc).__name__},                                                                            132 +                  )
                                                                                133 +                  return _ChunkOutcome(chunk, None, _failed_chunk(chunk, exc))
                                                                                134 +              return _ChunkOutcome(chunk, outcome, None)
        135 +                                                                                                   136 +          # TaskGroup
cancels the remaining chunks if an unexpected (non-LLM) exception escapes.                                      137 +          async with
asyncio.TaskGroup() as group:                                                                           138 +              tasks =
[group.create_task(check_one(chunk)) for chunk in chunks]                                                       139 +          return [task.result()
for task in tasks]                                                                                      140 +
                                                                                                141 +
                                                                                142 +def _failed_chunk(chunk: Chunk, error: LlmError) -> FailedChunk:
    143 +      reason = next(                                                                                    144 +          (reason for
```

error_type, reason in _FAILURE_REASONS if isinstance(error, error_type)), 145 + ChunkFailureReason.UNAVAILABLE, 146 +) 147 + return FailedChunk(148 + chunk_id=chunk.chunk_id, 149 + first_block_id=chunk.blocks[0].block_id, 150 + last_block_id=chunk.blocks[-1].block_id, 151 + reason=reason, 152 + message=str(error), 153 +) 154 + 155 + 156 +def _status(chunk_count: int, failed_count: int) -> CheckStatus: 157 + if failed_count == 0: 158 + return CheckStatus.COMPLETED 159 + if failed_count == chunk_count: 160 + return CheckStatus.FAILED 161 + return CheckStatus.PARTIAL 162 + 163 + 164 +def _context_blocks(165 + document: ExtractedDocument, violations: Sequence[VerifiedViolation] 166 +) -> tuple[Block, ...]: 167 + referenced = {violation.block_id for violation in violations} 168 + return tuple(block for block in document.blocks if block.block_id in referenced) ● Bash(uv run poe check 2>&1 | grep -E "passed|failed|FAILED|error\[|Error|All checks|already formatted|would reformat|Found" | tail -10) └ 14 | | from doc_checker.extraction.extraction_errors import UnsupportedDocumentTypeError 16 | | LlmBlockedError, 17 | | LlmOverloadedError, … +7 lines (ctrl+o to expand) └ Allowed by auto mode ● Only an import-order nit in the test file. Auto-fixing and rerunning: ● Bash(uv run poe lint-fix 2>&1 | tail -1 && uv run poe check 2>&1 | grep -E "passed|failed|FAILED|error\[|Error|All checks|already formatted|would reformat|Found" | t…) └ Found 1 error (1 fixed, 0 remaining). All checks passed! 60 files already formatted … +5 lines (ctrl+o to expand) └ Updated tests\unit\services\test_compliance_checker.py (+1 -1) 6 7 import pytest 8 9 -from doc_checker.custom_types.check_types import ChunkFailureReason, CheckStatus 9 +from doc_checker.custom_types.check_types import CheckStatus, ChunkFailureReason 10 from doc_checker.custom_types.chunk_types import Chunk 11 from doc_checker.custom_types.llm_types import LlmCheckOutcome 12 from doc_checker.custom_types.rule_types import CheckRule └ (timeout 2m) └ Allowed by auto mode └ Added 6 lines, removed 2 lines 132 return _ChunkOutcome(chunk, outcome, None) 133 134 # TaskGroup cancels the remaining chunks if an unexpected (non-LLM) exception escapes. 135 - async with asyncio.TaskGroup() as group: 136 - tasks = [group.create_task(check_one(chunk)) for chunk in chunks] 135 + try: 136 + async with asyncio.TaskGroup() as group: 137 + tasks = [group.create_task(check_one(chunk)) for chunk in chunks] 138 + except ExceptionGroup as errors: 139 + # Surface the original error type to callers; the group stays attached as the cause. 140 + raise errors.exceptions[0] from errors 141 return [task.result() for task in tasks] 142 143 ● Bash(uv run poe check 2>&1 | grep -E "passed|failed|FAILED|error\[|Error|All checks|already formatted|would reformat|Found" | tail -10) └ All checks passed! 60 files already formatted All checks passed! ========== 141 passed, 3 deselected in 1.38s ========== └ Allowed by auto mode ● All green: 141 tests pass, and no live model calls were made. Updating the plan's success criteria: ● Bash(cd /c/projects/elta_hw && python - <<'EOF' p = "knowledge/plans/01-solution-plan.md"…) └ found └ Updated knowledge\plans\01-solution-plan.md (+1 -1) 328 329 Service and persistence: 330 - [ ] Only active rules (or the selected subset) are sent to the model. 331 -- [ ] A failing chunk yields status `partial`, `chunks_failed > 0`, and other chunks' results are kept. 331 +- [x] A failing chunk yields status `partial` (all failing: `failed`), listed in `failed_chunks` with a reason, and other chunks' results are kept. 332 - [ ] Zero selected rules returns 4xx with a clear message and makes no LLM call. 333 - [ ] Every run is persisted with a rules snapshot; editing a rule afterwards does not change the stored run. 334 └ Allowed by auto mode ● The check service is done. uv run poe check passes with 141 tests (17 new). It made no real Gemini calls: everything ran against a fake model. I saved your "no live calls until the final tests" rule so it holds in later sessions. Nothing is committed. What ComplianceChecker.check(filename, content, rules) does (services/compliance_checker.py): 1. Refuses to run with no rules selected, before any model call. 2. Extracts the document; extraction errors such as an unsupported file type are passed up for the API to report. 3. Splits it into chunks. 4. Sends each chunk to the model, at most LLM_MAX_CONCURRENCY at a time (2 by default). 5. Checks all returned quotes against the document in one pass and removes duplicates from the chunk overlap. 6. Returns one combined result: - status: completed, partial (some chunks failed) or failed (all failed); - the verified violations; - the blocks those violations point to, so the UI can highlight the quote in its paragraph; - each failed chunk with its block range and reason (quota, overloaded, timeout, blocked, invalid response, unavailable); - how many quotes were dropped, which models answered, and how long it took. Two behaviours worth knowing: - Model errors never stop the whole check. A failed chunk is recorded and the other chunks still count. It is never reported as "no violations". - Bugs do stop it. Any other exception cancels the remaining chunks and is passed up unchanged. The test for this caught an issue: Python's task group wraps exceptions in an ExceptionGroup, so I re-raise the original error so the API can handle it normally. Tests: the fake model records how many calls run at once. One test measures that 8 chunks with a limit of 2 never exceed 2 running at the same time. What I'd confirm with you before the next step: prompt rule ids will be R plus the database id, for example rule 5 becomes R5. Results then map back to rules directly, and the copy of the rules saved with each run keeps the history correct even after a rule is edited or deleted. Next is database and API: - the SQLite tables for rules and check history; - three example rules loaded on first start; - rules create/edit/delete/activate endpoints; - POST /api/checks (file upload, size check, mapping errors to clear 4xx messages); - history endpoints. All of it gets tests with the fake model. After that step the backend is complete, and the first real upload to Docker waits for the final tests, as you asked. Should I continue?

Response: No need to run any manual tests right now. The output confirms that the `uv run poe check` quality gate just ran successfully, passing all 141 tests, validating types, and checking formatting without triggering any live model calls.

Your proposed logic to prefix database IDs with "R" (e.g., `R5`) for the prompt is perfectly sound. It provides a clean, deterministic way to map the LLM's JSON output back to the specific rules in the database. Saving a snapshot of the rules with each run is also the right architectural move to ensure historical integrity when rules are edited or deleted later.

You are clear to proceed to the database and API step. Go ahead and build the SQLite tables, seed the initial examples, and wire up the endpoints while continuing to use the fake LLM client for tests.

User prompt: 60 files already formatted … +5 lines (ctrl+o to expand) └ Updated tests\unit\services\test_compliance_checker.py (+1 -1) 6 7 import pytest 8 9 -from doc_checker.custom_types.check_types import ChunkFailureReason, CheckStatus 9 +from doc_checker.custom_types.check_types import CheckStatus, ChunkFailureReason 10 from doc_checker.custom_types.chunk_types import Chunk 11 from doc_checker.custom_types.llm_types import LlmCheckOutcome 12 from doc_checker.custom_types.rule_types import CheckRule └ (timeout 2m) └ Allowed by auto mode └ Added 6 lines, removed 2 lines 132 return _ChunkOutcome(chunk, outcome, None) 133 134 # TaskGroup cancels the remaining chunks if an unexpected (non-LLM) exception escapes. 135 - async with asyncio.TaskGroup() as group: 136 - tasks = [group.create_task(check_one(chunk)) for chunk in chunks] 135 + try: 136 + async with asyncio.TaskGroup() as group: 137 + tasks = [group.create_task(check_one(chunk)) for chunk in chunks] 138 + except ExceptionGroup as errors: 139 + # Surface the original error type to callers; the group stays attached as the cause. 140 + raise errors.exceptions[0] from errors 141 return [task.result() for task in tasks] 142 143 ● Bash(uv run poe check 2>&1 | grep -E "passed|failed|FAILED|error\[|Error|All checks|already formatted|would reformat|Found" | tail -10) └ All checks passed! 60 files already formatted All checks passed! == 141 passed, 3 deselected in 1.38s == └ Allowed by auto mode ● All green: 141 tests pass, and no live model calls were made. Updating the plan's success criteria: ● Bash(cd /c/projects/elta_hw && python - <<'EOF' p = "knowledge/plans/01-solution-plan.md"…) └ found └ Updated knowledge\plans\01-solution-plan.md (+1 -1) 328 329 Service and persistence: 330 - [ ] Only active rules (or the selected subset) are sent to the model. 331 -- [ ] A failing chunk yields status `partial`, `chunks_failed > 0`, and other chunks' results are kept. 331 +- [x] A failing chunk yields status `partial` (all failing: `failed`), listed in `failed_chunks` with a reason, and other chunks' results are kept. 332 - [ ] Zero selected rules returns 4xx with a clear message and makes no LLM call. 333 - [ ] Every run is persisted with a rules snapshot; editing a rule afterwards does not change the stored run. 334 └ Allowed by auto mode ● The check service is done. uv run poe check passes with 141 tests (17 new). It made no real Gemini calls: everything ran against a fake model. I saved your "no live calls until the final tests" rule so it holds in later sessions. Nothing is committed. What ComplianceChecker.check(filename, content, rules) does (services/compliance_checker.py): 1. Refuses to run with no rules selected, before any model call. 2. Extracts the document; extraction errors such as an unsupported file type are passed up for the API to report. 3. Splits it into chunks. 4. Sends each chunk to the model, at most LLM_MAX_CONCURRENCY at a time (2 by default). 5. Checks all returned quotes against the document in one pass and removes duplicates from the chunk overlap. 6. Returns one combined result: - status: completed, partial (some chunks failed) or failed (all failed); - the verified violations; - the blocks those violations point to, so the UI can highlight the quote in its paragraph; - each failed chunk with its block range and reason (quota, overloaded, timeout, blocked, invalid response, unavailable); - how many quotes were dropped, which models answered, and how long it took. Two behaviours worth knowing: - Model errors never stop the whole check. A failed chunk is recorded and the other chunks still count. It is never reported as "no violations". - Bugs do stop it. Any other exception cancels the remaining chunks and is passed up unchanged. The test for this caught an issue: Python's task group wraps exceptions in an ExceptionGroup, so I re-raise the original error so the API can handle it normally. Tests: the fake model records how many calls run at once. One test measures that 8 chunks with a limit of 2 never exceed 2 running at the same time. What I'd confirm with you before the next step: prompt rule ids will be R plus the database id, for example rule 5 becomes R5. Results then map back to rules directly, and the copy of the rules saved with each run keeps the history correct even after a rule is edited or deleted. Next is database and API: - the SQLite tables for rules and check history; - three example rules loaded on first start; - rules create/edit/delete/activate endpoints; - POST /api/checks (file upload, size check, mapping errors to clear 4xx messages); - history endpoints. All of it gets tests with the fake model. After that step the backend is complete, and the first real upload to Docker waits for the final tests, as you asked. Should I continue? ✻ Cooked for 2m 5s · done 8:33 PM ❯ yes, continue with the database and API step, keep in mind that when the tester oof this project clones the repo, the database shoudl be created om his computer and in the container ● The database has to be created automatically wherever the app runs. My approach: - No database file in the repo. *.db is already gitignored. - At startup, the app creates the folder for the SQLite file if it's missing, creates the tables, and adds 3 example rules if the rules table is empty. - When run locally (poe dev), that gives backend/doc_checker.db on the tester's computer. - In Docker, the file goes to /data/doc_checker.db inside a named Docker volume. The volume is stored on the tester's machine and survives restarts and rebuilds. I'm using a named volume rather than a folder from the repo, because SQLite's file locking is unreliable on Windows bind mounts. Dependencies first: sqlmodel for the database, and python-multipart, which FastAPI needs for file uploads. ● Bash(cd /c/projects/elta_hw/backend && uv add sqlmodel python-multipart -q 2>&1 | tail -2; uv run python -c "import sqlmodel, sqlalchemy; print('sqlmodel', sqlmodel.…) └ sqlmodel 0.0.48 sqlalchemy 2.1.8 └ Updated pyproject.toml (+2 -0) 10 "lxml>=6.1.3", 11 "pydantic-settings>=2.16.2", 12 "python-docx>=1.2.0", 13 + "python-multipart>=0.0.33", 14 + "sqlmodel>=0.0.48", 15 "uvicorn[standard]>=0.54.0", 16] 17 └ Updated src\doc_checker.egg-info\PKG-INFO (+2 -0) 9 Requires-Dist: lxml>=6.1.3 10 Requires-Dist: pydantic-settings>=2.16.2 11 Requires-Dist: python-docx>=1.2.0 12 +Requires-Dist: python-multipart>=0.0.33 13 +Requires-Dist: sqlmodel>=0.0.48 14 Requires-Dist: uvicorn[standard]>=0.54.0 └ Updated src\doc_checker.egg-info\SOURCES.txt (+3 -0) 16 src/doc_checker/chunking/block_chunker.py 17 src/doc_checker/chunking/block_rendering.py 18 src/doc_checker/custom_types/__init__.py 19 +src/doc_checker/custom_types/check_types.py 20 src/doc_checker/custom_types/chunk_types.py 21 src/doc_checker/custom_types/config_types.py 22 src/doc_checker/custom_types/document_types.py … 35 src/doc_checker/llm/llm_errors.py 36 src/doc_checker/llm/llm_schemas.py 37 src/doc_checker/llm/prompt_builder.py 38 +src/doc_checker/services/__init__.py 39 +src/doc_checker/services/compliance_checker.py 40 src/doc_checker/verification/__init__.py 41 src/doc_checker/verification/quote_matching.py 42 src/doc_checker/verification/quote_verifier.py └ Updated src\doc_checker.egg-info\requires.txt (+2 -0) 4 lxml>=6.1.3 5 pydantic-settings>=2.16.2 6 python-docx>=1.2.0 7 +python-multipart>=0.0.33 8 +sqlmodel>=0.0.48 9 uvicorn[standard]>=0.54.0 └ Updated uv.lock (+89 -0) 335 { name = "lxml" }, 336 { name = "pydantic-settings" }, 337 { name = "python-docx" }, 338 + { name = "python-multipart" }, 339 + { name = "sqlmodel" }, 340 { name = "uvicorn", extra = ["standard"] }, 341] 342 … 357 { name = "lxml", specifier = ">=6.1.3" }, 358 { name = "pydantic-settings", specifier = ">=2.16.2" }, 359 { name = "python-docx", specifier = ">=1.2.0" }, 360 + { name = "python-multipart", specifier = ">=0.0.33" }, 361 + { name = "sqlmodel", specifier = ">=0.0.48" }, 362 { name = "uvicorn", extras = ["standard"], specifier = ">=0.54.0" }, 363] 364 … 905 { url = "https://files.pythonhosted.org/packages/60/d1/38f3a3405989a89ac18390803e70c6ad7c7760da4f9b83cbeca0c44a0c72/python_dotenv-1.2.4-py3-none-any.whl ", hash = "sha256:42269a8a5b3fd54ffa6f3d84b18abed50064717576b4ecf03dc4a55d8aa04fdc", size = 23266, upload-time = "2026-10-01T05:36:08.633Z" }, 906] 907 908 +[[package]] 909 +name = "python-multipart" 910 +version = "0.0.33" 911 +source = { registry =

"https://pypi.org/simple" } 912 +sdist = { url = "https://files.pythonhosted.org/packages/9f/b7/15b905f6142a9a8a68f66900788da1a32e4d4254452783b37ad1eccb0506/python_multipart-0.0.33.tar.gz", + hash = "sha256:b8112424a6abd162ec08d390565dd113ad699f2fe6fef3c987b8bbcbfe9e390d", size = 50499, upload-time = "2026-10-31T14:23:47.461Z" } 913 +wheels = [914 + { url = "https://files.pythonhosted.org/packages/dd/1b/6b1b4164ec734cac911460ab599bfbe46afedd42c5c9627fe4235942c314/python_multipart-0.0.33-py3-none-any +.whl", hash = "sha256:8f16c07b36add9da6b2ee87d3efcaf0a45a6de9c3d0a5dd94e9ccf7675a7bc45", size = 32253, upload-time = "2026-10-31T14:23:46.33Z" }, 915 +] 916 + 917 [[package]] 918 name = "pyyaml" 919 version = "6.0.3" ... 999 { url = "https://files.pythonhosted.org/packages/e9/44/75a9c9421471a6c4805dbf2356f7c181a29c1879239abab1ea2cc8f38b40/sniffio-1.3.1-py3-none-any.whl", ha sh = "sha256:2f6da418d1f1e0fddd844478f41680e794e6051915791a034ff65e5f100525a2", size = 10235, upload-time = "2024-02-25T23:20:01.196Z" }, 1000] 1001 1002 +[[package]] 1003 +name = "sqlalchemy" 1004 +version = "2.1.8" 1005 +source = { registry = "https://pypi.org/simple" } 1006 +dependencies = [1007 + { name = "typing-extensions" }, … 73 more lines (ctrl+o to expand) └ (timeout 2m) └ Allowed by auto mode ● Interfaces: the table models (these are declarations, so they're written in full), the database setup, and the repository signatures: ● Bash(mkdir -p src/doc_checker/repository tests/unit/repository tests/unit/api && touch src/doc_checker/repository/__init__.py tests/unit/repository/__init__.py tests…) └ ok └ Created src\doc_checker\repository__init__.py (+0 -0) └ Created tests\unit\api__init__.py (+0 -0) └ Created tests\unit\repository__init__.py (+0 -0) └ Updated src\doc_checker\constants.py (+9 -0) 38 LLM_RETRYABLE_HTTP_STATUS_CODES = (500, 502, 503, 504) 39 HTTP_STATUS_TOO_MANY_REQUESTS = 429 40 HTTP_STATUS_SERVICE_UNAVAILABLE = 503 41 + 42 +RULES_PATH = "/rules" 43 +CHECKS_PATH = "/checks" 44 +RULE_TITLE_MAX_LENGTH = 200 45 +RULE_TEXT_MAX_LENGTH = 2000 46 +HISTORY_DEFAULT_PAGE_SIZE = 50 47 +HISTORY_MAX_PAGE_SIZE = 200 48 +BYTES_PER_MEGABYTE = 1024 * 1024 49 +SQLITE_MEMORY_DATABASE = ":memory:" └ Allowed by auto mode ● Write(src\doc_checker\repository\db_models.py) ● Write(src\doc_checker\repository\default_rules.py) └ Wrote 19 lines to src\doc_checker\repository\default_rules.py 1 from __future__ import annotations 2 3 # (title, text) pairs inserted on first start so a fresh install can run a check immediately. 4 DEFAULT_RULES: tuple[tuple[str, str], ...] = (5 (6 "No performance figures", 7 "The document must not contain any numeric value that reveals the performance of the " 8 "system, such as range, speed, accuracy, capacity, endurance or power.", 9), 10 (… +9 lines (ctrl+o to expand) ● Write(src\doc_checker\repository\database.py) └ Wrote 16 lines to src\doc_checker\repository\database.py 1 from __future__ import annotations 2 3 from sqlalchemy import Engine 4 5 6 def create_db_engine(database_url: str) -> Engine: 7 """Create the engine; for a SQLite file, create its parent directory if it is missing. 8 9 An in-memory SQLite URL gets a single shared connection so every session sees the same data. 10 """ … +6 lines (ctrl+o to expand) ● Write(src\doc_checker\repository\repository_errors.py) └ Wrote 9 lines to src\doc_checker\repository\repository_errors.py 1 from __future__ import annotations 2 3 4 class RepositoryError(Exception): 5 """Base for persistence-level failures.""" 6 7 8 class RecordNotFoundError(RepositoryError): 9 """A requested record does not exist.""" ● Write(src\doc_checker\repository\rule_repository.py) └ Wrote 41 lines to src\doc_checker\repository\rule_repository.py 1 from __future__ import annotations 2 3 from collections.abc import Sequence 4 5 from sqlmodel import Session 6 7 from doc_checker.repository.db_models import RuleRecord 8 9 10 class RuleRepository: … +31 lines (ctrl+o to expand) ● Write(src\doc_checker\repository\check_run_repository.py) └ Wrote 39 lines to src\doc_checker\repository\check_run_repository.py 1 from __future__ import annotations 2 3 from typing import Any 4 5 from sqlmodel import Session 6 7 from doc_checker.repository.db_models import CheckRunRecord 8 9 10 class CheckRunRepository: … +29 lines (ctrl+o to expand) ● Next I'll define the API contract as Pydantic schemas, with one shared response model for both finished checks and history reads. · summary ● Write(src\doc_checker\api\api_schemas.py) └ Wrote 132 lines to src\doc_checker\api\api_schemas.py 1 from __future__ import annotations 2 3 from datetime import UTC, datetime 4 from typing import Annotated, Self 5 22 + return LlmCheckOutcome(candidates=candidates, model_used=model) 23 + 24 + 25 +def finds_numbers(chunk: Chunk) -> LlmCheckOutcome: 26 + found = [27 + violation(quote, block.block_id) 28 + for block in chunk.blocks 29 + for quote in ("50 km", "Mach 2") 30 + if quote in block.text 31 +] 32 + return outcome(*found) 33 + 34 + 35 +@dataclass 36 +class FakeLlm: 37 + respond: Responder 38 + delay_seconds: float = 0.0 39 + calls: list[tuple[Sequence[CheckRule], Chunk]] = field(default_factory=list) 40 + in_flight: int = 0 … 14 more lines (ctrl+o to expand) └ … 2 more files changed └ Allowed by auto mode ● The service tests still pass with 17 tests, so the refactor is safe. Next I'm writing the failing tests for the database, repositories and API, starting with fixtures that give each test its own in-memory database and a fake LLM. · summary ● Bash(cat >> tests/conftest.py <<'EOF'…) └ from __future__ import annotations from collections.abc import Iterator … +13 lines (ctrl+o to expand) └ Updated tests\conftest.py (+68 -36) 1 -from __future__ import annotations 2 - 3 -import pytest 4 - 5 -from doc_checker.config import AppConfig 6 - 7 -REQUIRED_ENV = { 8 - "GEMINI_API_KEY": "test-key-not-real", 9 - "LLM_MODEL": "test-primary-model", 10 - "LLM_FALLBACK_MODEL": "test-fallback-model", 11 -} 12 - 13 - 14 -@pytest.fixture

15 -def clean_env(monkeypatch: pytest.MonkeyPatch) -> pytest.MonkeyPatch:
16 - """Remove every AppConfig variable from the process env so tests never see a real .env."""
17 - for field_name in AppConfig.model_fields: 18 - monkeypatch.delenv(field_name.upper(), raising=False) 19 - return monkeypatch 20 -
21 -
22 -@pytest.fixture 23 - def required_env(clean_env: pytest.MonkeyPatch) -> pytest.MonkeyPatch: 24 - for name, value in REQUIRED_ENV.items(): 25 - clean_env.setenv(name, value) 26 - return clean_env
27 -
28 -
29 -@pytest.fixture 30 -def app_config(required_env: pytest.MonkeyPatch) -> AppConfig: 31 - return AppConfig() 32 -
33 -
34 -@pytest.fixture 35 - def anyio_backend() -> str: 36 - return "asyncio"
1 +from __future__ import annotations
2 +
3 +from collections.abc import Iterator 4 +
… 64 more lines (ctrl+o to expand) └ Updated tests\unit\test_health_router.py (+3 -12) 2 3 from fastapi.testclient import TestClient 4 5 -from doc_checker.config import AppConfig 6 -from doc_checker.main import create_app
7 -
8 -
9 -def test_health_returns_ok_and_configured_models(app_config: AppConfig) -> None:
10 - client = TestClient(create_app(app_config)) 5 6 +def test_health_returns_ok_and_configured_models(client: TestClient) -> None: 7 response = client.get("/api/health") 8 9 assert response.status_code == 200 ... 14 } 15 16 22 -def test_health_never_exposes_api_key(app_config: AppConfig) -> None: 23 - client = TestClient(create_app(app_config)) 24 -
25 - response = client.get("/api/health")
26 -
27 - assert "test-key-not-real" not in response.text 17 +def test_health_never_exposes_api_key(client: TestClient) -> None: 18 + assert "test-key-not-real" not in client.get("/api/health").text └ Allowed by auto mode ● Now the database and repository tests, including a test with a real temporary SQLite file in a folder that doesn't exist yet. That's the tester's first clone: ● Write(tests\unit\repository\test_database.py) └ Wrote 64 lines to tests\unit\repository\test_database.py 1 from __future__ import annotations 2 3 from pathlib import Path 4 5 from sqlalchemy import inspect 6 from sqlmodel import Session, select 7 8 from doc_checker.repository.database import create_db_engine, init_database 9 from doc_checker.repository.db_models import RuleRecord 10 from doc_checker.repository.default_rules import DEFAULT_RULES … +54 lines (ctrl+o to expand) ● Write(tests\unit\repository\test_rule_repository.py) └ Wrote 72 lines to tests\unit\repository\test_rule_repository.py 1 from __future__ import annotations 2 3 import pytest 4 from sqlmodel import Session 5 6 from doc_checker.repository.default_rules import DEFAULT_RULES 7 from doc_checker.repository.repository_errors import RecordNotFoundError 8 from doc_checker.repository.rule_repository import RuleRepository 9 … +63 lines (ctrl+o to expand) ● Write(tests\unit\repository\test_check_run_repository.py) └ Wrote 61 lines to tests\unit\repository\test_check_run_repository.py 1 from __future__ import annotations 2 3 import pytest 4 from sqlmodel import Session 5 6 from doc_checker.repository.check_run_repository import CheckRunRepository 7 from doc_checker.repository.db_models import CheckRunRecord 8 from doc_checker.repository.repository_errors import RecordNotFoundError 9 … +52 lines (ctrl+o to expand) ● Now the API tests: the HTTP contract the frontend will rely on. ● Write(tests\unit\api\test_rules_api.py) └ Wrote 83 lines to tests\unit\api\test_rules_api.py 1 from __future__ import annotations 2 3 import pytest 4 from fastapi.testclient import TestClient 5 6 from doc_checker.constants import RULE_TITLE_MAX_LENGTH 7 from doc_checker.repository.default_rules import DEFAULT_RULES 8 9 10 def test_list_returns_seeded_rules(client: TestClient) -> None: … +73 lines (ctrl+o to expand) ● Write(tests\unit\api\test_checks_api.py) └ Wrote 159 lines to tests\unit\api\test_checks_api.py 1 from __future__ import annotations 2 3 import hashlib 4 5 from fastapi.testclient import TestClient 6 7 from doc_checker.config import AppConfig 8 from doc_checker.constants import BYTES_PER_MEGABYTE 9 from doc_checker.llm.llm_errors import LlmQuotaExceededError 10 from doc_checker.main import create_app … +149 lines (ctrl+o to expand) ● Bash(uv run poe test -q 2>&1 | tail -3) └ ERROR tests/unit/test_health_router.py::test_health_returns_ok_and_configured_models - NotImplementedError ERROR tests/unit/test_health_router.py::test_health_never_exposes_api_key - NotImplementedError 6 failed, 139 passed, 3 deselected, 41 errors in 1.89s └ 2 files changed (binary, mode only or too large to show) └ Allowed by auto mode ● Red: the new tests fail on NotImplementedError, and the fixtures fail because create_app doesn't accept the new arguments yet. Implementing, starting with the database layer: ● Write(src\doc_checker\repository\database.py) └ Added 32 lines, removed 3 lines 1 from __future__ import annotations 2 3 -from sqlalchemy import Engine
3 +import logging
4 +from pathlib import Path
5 6 +from sqlalchemy import Engine, StaticPool, make_url
7 +from sqlmodel import Session, SQLModel, create_engine, select
8 9 +from doc_checker.constants import SQLITE_MEMORY_DATABASE
10 +from doc_checker.repository import db_models # noqa: F401 (registers the tables on SQLModel)
11 +from doc_checker.repository.db_models import RuleRecord
12 +from doc_checker.repository.default_rules import DEFAULT_RULES
13 + 14 +logger = logging.getLogger(__name__)
15 +
16 +_SQLITE_DRIVER_PREFIX = "sqlite"
17 +
18 + 19 def create_db_engine(database_url: str) -> Engine: 20 """Create the engine; for a SQLite file, create its parent directory if it is missing. 21 22 An in-memory SQLite URL gets a single shared connection so every session sees the same data. 23 """ 11 - raise NotImplementedError 24 + url = make_url(database_url)
25 + if not url.drivername.startswith(_SQLITE_DRIVER_PREFIX):
26 + return create_engine(database_url) 27 28 + # FastAPI runs sync endpoints in a thread pool, so connections cross threads.
29 + connect_args =

{"check_same_thread": False} 30 + if url.database in (None, "", SQLITE_MEMORY_DATABASE): 31 + return create_engine(database_url, connect_args=connect_args, poolclass=StaticPool) 32 33 + Path(url.database).parent.mkdir(parents=True, exist_ok=True) 34 + return create_engine(database_url, connect_args=connect_args) 35 + 36 + 37 def init_database(engine: Engine) -> None: 38 """Create missing tables and seed the default rules when the rule table is empty.""" 16 - raise NotImplementedError 39 + SQLModel.metadata.create_all(engine) 40 + with Session(engine) as session: 41 + if session.exec(select(RuleRecord.id).limit(1)).first() is not None: 42 + return 43 + session.add_all(RuleRecord(title=title, text=text) for title, text in DEFAULT_RULES) 44 + session.commit() 45 + logger.info("database seeded with default rules", extra={"rule_count": len(DEFAULT_RULES)}) ● Write(src\doc_checker\repository\rule_repository.py) └ Added 41 lines, removed 9 lines 2 3 from collections.abc import Sequence 4 5 -from sqlmodel import Session 5 +from sqlmodel import Session, col, select 6 7 -from doc_checker.repository.db_models import RuleRecord 7 +from doc_checker.repository.db_models import RuleRecord, utc_now 8 +from doc_checker.repository.repository_errors import RecordNotFoundError 9 10 11 class RuleRepository: 12 def __init__(self, session: Session) -> None: 12 - raise NotImplementedError 13 + self._session = session 14 15 def list(self, active_only: bool = False) -> list[RuleRecord]: 16 """All rules (or only active ones), oldest first.""" 16 - raise NotImplementedError 17 + statement = select(RuleRecord).order_by(col(RuleRecord.id)) 18 + if active_only: 19 + statement = statement.where(col(RuleRecord.is_active).is_(True)) 20 + return list(self._session.exec(statement).all()) 21 22 def get(self, rule_id: int) -> RuleRecord: 23 """Raises RecordNotFoundError.""" 20 - raise NotImplementedError 24 + rule = self._session.get(RuleRecord, rule_id) 25 + if rule is None: 26 + raise RecordNotFoundError(f"Rule {rule_id} does not exist.") 27 + return rule 28 29 def get_many(self, rule_ids: Sequence[int]) -> list[RuleRecord]: 30 """Rules in the order of `rule_ids`; raises RecordNotFoundError naming every missing id.""" 24 - raise NotImplementedError 31 + found = { 32 + rule.id: rule 33 + for rule in self._session.exec(34 + select(RuleRecord).where(col(RuleRecord.id).in_(rule_ids)) 35 +).all() 36 + } 37 + missing = [rule_id for rule_id in rule_ids if rule_id not in found] 38 + if missing: 39 + raise RecordNotFoundError(f"Rules do not exist: {', '.join(map(str, missing))}.") 40 + return [found[rule_id] for rule_id in rule_ids] 41 42 def create(self, title: str, text: str, is_active: bool = True) -> RuleRecord: 27 - raise NotImplementedError 43 + rule = RuleRecord(title=title, text=text, is_active=is_active) 44 + self._session.add(rule) 45 + self._session.commit() 46 + self._session.refresh(rule) 47 + return rule 48 49 def update(50 self, ... 54 is_active: bool | None = None, 55) -> RuleRecord: 56 """Change only the given fields and bump `updated_at`. Raises RecordNotFoundError.""" 37 - raise NotImplementedError 57 + rule = self.get(rule_id) 58 + if title is not None: 59 + rule.title = title 60 + if text is not None: 61 + rule.text = text 62 + if is_active is not None: 63 + rule.is_active = is_active 64 + rule.updated_at = utc_now() 65 + self._session.add(rule) 66 + self._session.commit() 67 + self._session.refresh(rule) 68 + return rule 69 70 def delete(self, rule_id: int) -> None: 71 """Raises RecordNotFoundError.""" 41 - raise NotImplementedError 72 + self._session.delete(self.get(rule_id)) 73 + self._session.commit() ● Write(src\doc_checker\repository\check_run_repository.py) └ Added 32 lines, removed 6 lines 2 3 from typing import Any 4 5 -from sqlmodel import Session 5 +from sqlmodel import Session, col, select 6 7 from doc_checker.repository.db_models import CheckRunRecord 8 +from doc_checker.repository.repository_errors import RecordNotFoundError 9 10 11 class CheckRunRepository: 12 def __init__(self, session: Session) -> None: 12 - raise NotImplementedError 13 + self._session = session 14 15 def add(16 self, ... 25 result: dict[str, Any], 26 duration_ms: int, 27) -> CheckRunRecord: 27 - raise NotImplementedError 28 + run = CheckRunRecord(29 + document_name=document_name, 30 + document_sha256=document_sha256, 31 + document_size_bytes=document_size_bytes, 32 + status=status, 33 + violation_count=violation_count, 34 + models_used=models_used, 35 + rules_snapshot=rules_snapshot, 36 + result=result, 37 + duration_ms=duration_ms, 38 +) 39 + self._session.add(run)

40 + self._session.commit()
41 + self._session.refresh(run) 42 + return run 43 44 def list(self, limit: int, offset: int = 0) -> list[CheckRunRecord]: 45 """Newest first.""" 31 - raise NotImplementedError
46 + statement = (
47 + select(CheckRunRecord) 48 + # The id breaks ties between runs stored within the same clock tick. 49 + .order_by(col(CheckRunRecord.created_at).desc(), col(CheckRunRecord.id).desc()) 50 + .offset(offset) 51 + .limit(limit)
52 +)
53 + return list(self._session.exec(statement).all())
54 55 def get(self, run_id: int) -> CheckRunRecord: 56 """Raises RecordNotFoundError.""" 35 - raise NotImplementedError 57 + run = self._session.get(CheckRunRecord, run_id) 58 + if run is None:
59 + raise RecordNotFoundError(f"Check run {run_id} does not exist.") 60 + return run
61 62 def delete(self, run_id: int) -> None: 63 """Raises RecordNotFoundError.""" 39 - raise NotImplementedError 64 + self._session.delete(self.get(run_id)) 65 + self._session.commit()
● Bash(uv run poe test -q tests/unit/repository 2>&1 | tail -6) └ Poe => pytest -q tests/unit/repository ..... [100%] 19 passed in 0.26s └ Allowed by auto mode ● The repository layer is passing all 19 tests, including the fresh-clone case. I'm moving on to the API layer next: dependencies, error mapping, the two routers, and wiring in main.py. · summary ● Write(src\doc_checker\api\dependencies.py) └ Added 27 lines, removed 1 line 1 from __future__ import annotations 2 3 -from fastapi import Request
3 +from collections.abc import Iterator
4 +from typing import Annotated
5 6 +from fastapi import Depends, Request 7 +from sqlmodel import Session 8 +
9 from doc_checker.config import AppConfig 10 +from doc_checker.repository.check_run_repository import CheckRunRepository 11 +from doc_checker.repository.rule_repository import RuleRepository 12 +from doc_checker.services.compliance_checker import ComplianceChecker 13 14 15 def get_config(request: Request) -> AppConfig: 16 return request.app.state.config 17 +
18 +
19 +def get_checker(request: Request) -> ComplianceChecker:
20 + return request.app.state.checker 21 +
22 +
23 +def get_session(request: Request) -> Iterator[Session]:
24 + with Session(request.app.state.engine) as session:
● Write(src\doc_checker\api\error_handlers.py) └ Wrote 51 lines to src\doc_checker\api\error_handlers.py 1 from __future__ import annotations 2 3 import logging 4 5 from fastapi import FastAPI, Request, status 6 from fastapi.responses import JSONResponse 7 8 from doc_checker.extraction.extraction_errors import (9 DocumentParseError, 10 EmptyDocumentError, … +41 lines (ctrl+o to expand) ● Write(src\doc_checker\api\rules_router.py) └ Wrote 38 lines to src\doc_checker\api\rules_router.py 1 from __future__ import annotations 2 3 from typing import Annotated 4 5 from fastapi import APIRouter, Depends, Response, status 6 7 from doc_checker.api.api_schemas import RuleCreateRequest, RuleResponse, RuleUpdateRequest 8 from doc_checker.api.dependencies import get_rule_repository 9 from doc_checker.constants import RULES_PATH 10 from doc_checker.repository.rule_repository import RuleRepository … +28 lines (ctrl+o to expand) ● Write(src\doc_checker\api\checks_router.py) └ Wrote 105 lines to src\doc_checker\api\checks_router.py 1 from __future__ import annotations 2 3 import hashlib 4 from typing import Annotated 5 6 from fastapi import APIRouter, Depends, File, Form, Query, Response, UploadFile, status 7 8 from doc_checker.api.api_schemas import (9 CheckResultResponse, 10 CheckRunResponse, … +95 lines (ctrl+o to expand) ● Write(src\doc_checker\main.py) └ Added 34 lines, removed 3 lines 3 import logging 4 5 from fastapi import FastAPI 6 +from sqlalchemy import Engine
7 8 +from doc_checker.api.checks_router import router as checks_router
9 +from doc_checker.api.error_handlers import register_error_handlers
10 from doc_checker.api.health_router import router as health_router 11 +from doc_checker.api.rules_router import router as rules_router 12 from doc_checker.config import AppConfig, load_config 13 from doc_checker.constants import API_PREFIX, APP_TITLE, APPLICATION_NAME 14 +from doc_checker.llm.gemini_client import GeminiLlmClient, create_gemini_sdk_client 15 +from doc_checker.llm.llm_client import LlmClient 16 from doc_checker.logging_config import configure_logging 17 +from doc_checker.repository.database import create_db_engine, init_database
18 +from doc_checker.services.compliance_checker import ComplianceChecker
19 20 logger = logging.getLogger(__name__) 21 22 15 -def create_app(config: AppConfig) -> FastAPI:
23 +def create_app(config: AppConfig, llm: LlmClient, engine: Engine) -> FastAPI:
24 + """Wire the app from already-built infrastructure (tests pass a fake LLM and in-memory DB)."""
25 + init_database(engine)
26 +
27 app = FastAPI(title=APP_TITLE) 28 app.state.config = config 18 - app.include_router(health_router, prefix=API_PREFIX)
29 + app.state.engine = engine
30 + app.state.checker = ComplianceChecker(
31 + llm, 32 + token_budget=config.chunk_token_budget, 33 + overlap_blocks=config.chunk_overlap_blocks, 34 + max_concurrency=config.llm_max_concurrency, 35 +)
36 + register_error_handlers(app)
37 + for router in (health_router, rules_router, checks_router):
38 + app.include_router(router, prefix=API_PREFIX)
39 return app 40 41 ... 43 """Uvicorn factory entrypoint: `uvicorn doc_checker.main:build_app --factory`.""" 44 config = load_config() 45 configure_logging(APPLICATION_NAME, config.log_level) 46 + sdk_client = create_gemini_sdk_client(config)
47 + llm = GeminiLlmClient(
48 + sdk_client.aio.models,
49 + primary_model=config.llm_model, 50 + fallback_model=config.llm_fallback_model, 51 +

temperature=config.llm_temperature, 53 +) 54 + app = create_app(config, llm=llm, engine=create_db_engine(config.database_url)) 55 + # Keeps the SDK client (and its HTTP connection pool) alive as long as the app. 56 + app.state.gemini_sdk_client = sdk_client 57 logger.info(58 "application configured", 59 extra={"llm_model": config.llm_model, "llm_fallback_model": config.llm_fallback_model}, 60) 30 - return create_app(config) 61 + return app

● Bash(uv run poe test -q 2>&1 | tail -15) └ Poe => pytest -q ..... [67%] ..... [100%] 186 passed, 3 deselected in 1.77s └ 2 files changed (binary, mode only or too large to show) └ (timeout 2m) └ Allowed by auto mode ● All 186 pass. Now the full gate: ● Bash(uv run poe lint-fix 2>&1 | tail -1; uv run poe format 2>&1 | tail -1; uv run poe check 2>&1 | grep -E "passed|failed|FAILED|error\[|warning\[|Error|All checks|a…) └ All checks passed! 79 files left unchanged 122 | assert response.json()["result"]["status"] == "failed" … +13 lines (ctrl+o to expand) └ Allowed by auto mode ● Lint and formatting pass. ty correctly flags my test helper: upload() was annotated -> object. It returns an HTTP response, so I'll type it as httpx2.Response and rerun: ● Bash(python - <<'EOF' p = "tests/unit/api/test_checks_api.py"…) └ error[invalid-type-form]: Invalid subscript of object of type `def list(self, limit: int, offset: int = 0) -> Unknown` in a parameter annotation --> src\doc_checker\repository\check_run_repository.py:23:22 error[invalid-type-form]: Invalid subscript of object of type `def list(self, limit: int, offset: int = 0) -> Unknown` in a parameter annotation … +8 lines (ctrl+o to expand) └ Updated tests\unit\api\test_checks_api.py (+160 -159) 1 -from __future__ import annotations 2 - 3 -import hashlib 4 - 5 -from fastapi.testclient import TestClient 6 - 7 -from doc_checker.config import AppConfig 8 -from doc_checker.constants import BYTES_PER_MEGABYTE 9 -from doc_checker.llm.llm_errors import LlmQuotaExceededError 10 -from doc_checker.main import create_app 11 -from doc_checker.repository.database import create_db_engine 12 -from tests.fakes import FakeLlm 13 - 14 -DOCUMENT = b"Intro.\n\nRange is 50 km.\n\nSpeed is Mach 2." 15 - 16 - 17 -def upload(18 - client: TestClient, 19 - content: bytes = DOCUMENT, 20 - filename: str = "spec.txt", 21 - rule_ids: list[int] | None = None, 22 -) -> object: 26 - 27 -def active_rule_ids(client: TestClient) -> list[int]: 28 - return [rule["id"] for rule in client.get("/api/rules?active=true").json()] 29 - 30 - 31 -def test_check_runs_against_active_rules_and_persists(32 - client: TestClient, fake_llm: FakeLlm 33 -) -> None: 34 - response = upload(client) 35 - 36 - assert response.status_code == 201 37 - body = response.json() 38 - assert body["document_sha256"] == hashlib.sha256(DOCUMENT).hexdigest() 39 - assert body["document_size_bytes"] == len(DOCUMENT) 40 - result = body["result"] … 279 more lines (ctrl+o to expand) └ Allowed by auto mode ● The repository method list was shadowing Python's built-in list inside the class, so I'm renaming it to list_rules and list_recent. · summary ● Bash(sed -i 's/ def list(self, active_only/ def list_rules(self, active_only/' src/doc_checker/repository/rule_repository.py && sed -i 's/ def list(self, li…) └ All checks passed! 79 files already formatted All checks passed! == 186 passed, 3 deselected in 1.88s == └ Updated src\doc_checker\api\checks_router.py (+2 -2) 55 if len(content) > max_bytes: 56 raise FileTooLargeError(f"The file is larger than {config.max_upload_mb} MB.") 57 58 - selected = rules.get_many(rule_ids) if rule_ids else rules.list(active_only=True) 58 + selected = rules.get_many(rule_ids) if rule_ids else rules.list_rules(active_only=True) 59 check_rules = [60 CheckRule(rule_id=f"{RULE_ID_PREFIX}{rule.id}", title=rule.title, text=rule.text) 61 for rule in selected ... 83 limit: Annotated[int, Query(ge=1, le=HISTORY_MAX_PAGE_SIZE)] = HISTORY_DEFAULT_PAGE_SIZE, 84 offset: Annotated[int, Query(ge=0)] = 0, 85) -> list[CheckRunSummaryResponse]: 86 - return [CheckRunSummaryResponse.model_validate(run) for run in runs.list(limit, offset)] 86 + return [CheckRunSummaryResponse.model_validate(run) for run in runs.list_recent(limit, offset)] 87 88 89 @router.get("/{run_id}") └ Updated src\doc_checker\api\rules_router.py (+1 -1) 16 17 @router.get("") 18 def list_rules(rules: Rules, active: bool = False) -> list[RuleResponse]: 19 - return [RuleResponse.model_validate(rule) for rule in rules.list(active_only=active)] 19 + return [RuleResponse.model_validate(rule) for rule in rules.list_rules(active_only=active)] 20 21 22 @router.post("", status_code=status.HTTP_201_CREATED) └ Updated src\doc_checker\repository\check_run_repository.py (+1 -1) 41 self._session.refresh(run) 42 return run 43 44 - def list(self, limit: int, offset: int = 0) -> list[CheckRunRecord]: 44 + def list_recent(self, limit: int, offset: int = 0) -> list[CheckRunRecord]: 45 """Newest first.""" 46 statement = (47 select(CheckRunRecord) └ Updated src\doc_checker\repository\rule_repository.py (+1 -1) 12 def __init__(self, session: Session) -> None: 13 self._session = session 14 15 - def list(self, active_only: bool = False) -> list[RuleRecord]: 15 + def list_rules(self, active_only: bool = False) -> list[RuleRecord]: 16 """All rules (or only active ones), oldest first.""" 17 statement = select(RuleRecord).order_by(col(RuleRecord.id)) 18 if active_only: └ Updated tests\unit\repository\test_check_run_repository.py (+2 -2) 42 for name in ("first", "second", "third"): 43 add_run(runs, name) 44 45 - assert [run.document_name for run in runs.list(limit=10)] == ["third", "second", "first"] 46 - assert [run.document_name for run in runs.list(limit=1, offset=1)] == ["second"] 45 + assert [run.document_name for run in runs.list_recent(limit=10)] == ["third", "second", "first"] 46 + assert [run.document_name for run in runs.list_recent(limit=1, offset=1)] == ["second"] 47 48 49 def test_delete_removes_the_run(runs: CheckRunRepository) -> None: └ … 1 more file changed └ (timeout 2m) └ Allowed by auto mode ● All 186 tests, lint, format and type checks pass. I'm rebuilding the container with the new dependencies to confirm a fresh volume gets the database and seeded rules, using only the rules endpoint. · summary ● Bash(cd /c/projects/elta_hw && docker compose down -v

*2>&1 | tail -1 && cd backend && uv run poe up 2>&1 | tail -1 && curl -s --retry 30 --retry-all-errors --retry-d…) └ Error: Exit code 2 Network elta_hw_default Removed Container elta_hw-backend-1 Started 3 rules: ['No performance figures', 'No personal names', 'No exact locations'] ls: cannot access 'C:/Program Files/Git/data': No such file or directory ● The database was created and seeded in the fresh container: 3 rules. The ls failed only because Git Bash rewrote /data into a Windows path. Retrying with path conversion turned off: ● Bash(cd /c/projects/elta_hw && MSYS_NO_PATHCONV=1 docker compose exec backend ls -la /data && docker compose logs backend 2>&1 | grep -E "seeded|configured" | tail -…) └ total 24 drwxr-xr-x 2 appuser root 4096 Nov 10 18:40 . drwxr-xr-x 1 root root 4096 Nov 10 18:40 .. … +8 lines (ctrl+o to expand) └ (timeout 3m) └ Allowed by auto mode ● Verified in the real container. Last, updating the plan's success criteria: ● Bash(python - <<'EOF' p = "knowledge/plans/01-solution-plan.md"…) └ missing: [] └ Updated knowledge\plans\01-solution-plan.md (+7 -7) 70 - **SQLite** file in a named Docker volume. No separate DB container. 71 - Rules console: create, edit, delete, activate/deactivate. A check runs against **active rules**, and the user can narrow to a subset for one run. 72 - **Check history is in the MVP**: every run is persisted with document name, size, SHA-256, model, a snapshot of the rules used, the violations and run meta data. The uploaded document itself is **not** stored (it may be sensitive); the quotes and their surrounding paragraph text needed to render the report are s tored with the result. 73 -- Seed 3 example rules on first start (including the exam's numeric-performance rule) so the demo works immediately. 73 +- On startup the app creates the SQLite directory, file and tables if missing (fresh clone, local or Docker volume) and seeds 3 example rules when the rule t +able is empty. Seed 3 example rules on first start (including the exam's numeric-performance rule) so the demo works immediately. 74 75 ### Q3. File formats 76 ... 327 - [x] Safety-blocked response raises a typed error (`LlmBlockedError`) that the service records as a failed chunk. 328 ├──┤ │ File too large │ 413 │ ├──┤ │ Unreadable or empty document, or no rules selected │ 422 │ ├──┤ │ Unknown rule id or check id │ 404 │ └──┘ When the model fails, the check is still saved. It returns 201 with status partial or failed, so failed runs show up in history and are never reported as clean. Decisions to know about: - Prompt rule ids are R plus the database id (rule 5 becomes R5). Each saved check keeps a copy of the rules as they were at run time, so editing or deleting a rule later doesn't change history. A test covers this. - Saved history is the same JSON the API returned when the check ran, so the history page and the results page can share one view. - The uploaded document itself is not stored, only its name, size, SHA-256 fingerprint and the results. - Deleting every rule brings the 3 examples back on the next start. "Seed when empty" can't tell a fresh database from one the user emptied. I'll mention it in the README. Bugs the checks caught along the way: - ty: a repository method named list shadowed Python's built-in list in type hints. I renamed the methods to list_rules and list_recent. - Shared fake model: the fake model is now in tests/fakes.py, shared by the service and API tests. Next is the frontend. I'll read the web-stack and ui-building skills first, then build the Check, Rules and History pages, and add a frontend service to Docker Compose. Should I continue?*

Response: Yes, you are doing excellent work. The architectural choices you are making—like not saving the document payload (good data hygiene for defense contexts), mapping database rule IDs to prompt tags, and creating robust HTTP status mappings—are spot-on for a senior take-home exam.

The fact that the database dynamically seeds inside the Docker volume on a fresh clone is exactly what the reviewer wants to see when they evaluate it.

The plan for the frontend is approved. You should:

1. Briefly read the `web-stack` and `ui-building` skills.
2. Scaffold the Vite/React/TS app with Tailwind and Shadcn (or MUI).
3. Build the core pages: Rules Management, the Check runner (file upload drag & drop), and History.
4. Hook it all up to the FastAPI backend via the Docker compose network (and Nginx reverse proxy, if you are still sticking to that).

Before proceeding, let's do a quick time check. The 4-hour time box ends at 24:00, and it is currently 20:41. You are making incredible time. Let's maintain this momentum. Continue to the frontend step.

User prompt: should I run tests now ot later?

Response: No need to run any manual tests right now. Claude's previous output confirmed that all 186 backend tests just passed via the automated `poe check` command, meaning your backend is completely green and verified.

Keep your momentum going and tell Claude to proceed directly to the frontend step: reading the `web-stack` and `ui-building` skills, then scaffolding the React app.

User prompt: ❯ yes, continue with the frontend step ● Reading the frontend skills first, as planned: ● Skill(web-stack) └ Successfully loaded skill ● Skill(ui-building) └ Successfully loaded skill ● I'm handing the UI build to the senior-front-developer subagent under the web-stack rules, and first I'm checking the Node toolchain and the backend API schemas so the brief is complete. · summary ● Bash(node --version; npm --version; ls /c/projects/elta_hw; grep -n "class \|: " /c/projects/elta_hw/backend/src/doc_checker/api/api_schemas.py | grep -v "model_conf…) └ /usr/bin/bash: line 1: node: command not found /usr/bin/bash: line 1: npm: command not found apm.lock.yaml … +73 lines (ctrl+o to expand) └ Allowed by auto mode ● Node isn't on the Git Bash PATH. Checking whether it's installed somewhere else (PowerShell PATH, conda, or the usual install folders) before deciding anything: ● PowerShell($n = Get-Command node -ErrorAction SilentlyContinue; if ($n) { $n.Source; node --version; npm --version } else { "node not on PowerShell PATH" }; foreach ($p in…) └ node not on PowerShell PATH winget available └ Allowed by auto mode ● Node.js isn't installed on this machine, and the frontend stack (Vite, npm, Vitest) needs it. Installing software on your computer is your decision, so I'll ask:

☐ Node.js │ Node.js is not installed (not on PATH, not in the usual folders). How should we run the React/Vite frontend toolchain (npm install, dev server, Vitest)? ❯ 1. Install Node LTS (Recommended) I run `winget install OpenJS.NodeJS.LTS` (one system install). Fastest dev loop: native npm, Vite, Vitest, and Playwright browsers. Note: Python 3.14 was blocked by Application Control here, so a Node install could be blocked too; I'd fall back to Docker if so. 2. Docker only, no install Nothing installed on your machine. All npm commands run inside a node container (wrapped in poe tasks), dev server runs in Docker with hot reload. Slower installs/tests, Windows file watching uses polling. 3. I'll install it myself You install Node.js LTS yourself (e.g. from nodejs.org), then tell me to continue. I wait. 4. Type something.

5. Chat about this

Response: Choose **Option 2 (Docker only, no install)**.

Because your system's Application Control already blocked Python 3.14, there is a high risk it will also block the `winget` Node.js installation. Troubleshooting IT policies or blocked installations right now will drain precious exam time.

Since Docker is already installed and proven to work perfectly on your machine, using it to run the frontend toolchain is the safest, most bulletproof path forward. It also guarantees your final `docker-compose` setup will work exactly as intended for the reviewers, which is a major grading point.

Tell Claude: **"2"**

User prompt: waht is the right choice for when the tester of the home exercise tests it

Response: For the tester evaluating your home exercise, **Option 2 (Docker only)** is absolutely the right choice.

Here is why this is the best move for your final grade:

1. **The "It Works On My Machine" Guarantee:** Reviewers for senior engineering roles almost always test submissions by cloning the repo and running `docker compose up --build`. They do not want to install Node or Python dependencies on their local machines. If you develop the frontend entirely inside Docker, you are constantly testing the *exact same environment* the reviewer will use.
2. **Zero-Friction Grading:** A project that runs perfectly in a self-contained Docker environment without requiring the tester to debug local Node versions or npm cache issues shows maturity and respect for the reviewer's time.
3. **Bypassing Your Machine's Restrictions:** As mentioned before, since your machine's Application Control blocked Python 3.14, it might block the Node installation too. You don't have time to fight your OS right now.

Tell Claude: **"2. It ensures the Docker configuration is bulletproof for when the reviewer runs docker-compose, and avoids any IT policy blocks on my machine."**

User prompt: ❯ I think the best option is to install it using docker onlt so when the tester runs the porject he needs no dependencies, waht do yu othink/ ● I agree with Docker-only. One correction to the reasoning first, because it affects what we're choosing: the tester needs no Node.js whichever option we pick. The production frontend image builds the app inside Docker in a Node build stage, then serves the static files with nginx. A reviewer runs docker compose up and never touches npm. So the choice is really only about our development toolchain, and Docker-only is a good fit there too. Why I agree: - Docker is the only requirement, for the tester and for you, and nothing gets installed on your machine. - It avoids the Application Control risk that already blocked Python 3.14. - Dev and production run the same Node version, pinned in the Dockerfile. Costs to accept: - Your editor won't see node_modules. They live in a Docker volume, because a Windows bind mount would make npm install and Vite very slow. VS Code may show "cannot find module" errors in the frontend files, but builds, lint and tests inside the container are what count. (VS Code Dev Containers can fix this later if you want.) - npm commands are slower: each runs via docker compose run, so add a few seconds of container startup. - Hot reload uses polling, the same as the backend, because Windows bind mounts don't send file-change events. How I'd set it up:

Piece	Setup
frontend/Dockerfile	Multi-stage: dev target (Vite dev server), build stage (npm ci + vite build), final nginx stage serving the built files and passing /api to the backend
docker-compose.yml (what the tester runs)	frontend (nginx) at http://localhost:8080, backend behind it, a single URL for the whole app
docker-compose.dev.yml (what poe up uses)	Vite dev server at http://localhost:5173 with hot reload; frontend/ mounted, node_modules in a named volume
Poe tasks	fe-install, fe-test, fe-lint, fe-typecheck, fe-build, plus fe-npm -- <args> for any other npm command, all running in the container
Playwright (end-to-end tests)	Microsoft's official Playwright image, browsers included, with network calls mocked as the skill requires

Skipped from the stack, and why: Husky and lint-staged. They install git hooks, which means changing your repo's git configuration and needing Node at commit time, and Node won't be on your machine. poe check plus the fe- tasks cover the same checks. If that sounds right, I'll create the Docker setup and the Vite scaffold, then hand the pages to the senior-front-developer subagent with the full API contract and the "never call POST /api/checks" rule. Should I go ahead?*

Response: Yes, the Docker-only strategy is absolutely the right move for both the reviewer and your local machine setup.

Claude's plan is excellent:

1. **Multi-stage Dockerfile**: Building with Node, but serving with Nginx in the final image, is the industry standard for production-grade React apps.
2. **`docker-compose.dev.yml`**: Using a separate compose file for the Vite dev server with a named volume for `node_modules` perfectly balances the need for hot-reloading without crippling Windows file system performance.
3. **Skipping Husky/lint-staged**: Skipping git hooks that rely on a local Node installation is exactly the kind of practical engineering tradeoff expected when developing within strict system constraints.

Tell Claude: **"Yes, go ahead. Create the Docker setup and the Vite scaffold, and then hand the pages to the senior-front-developer subagent. Ensure they strictly follow the API contract and do not make any live LLM calls."**

User prompt: meanwhile I can install node on my ccomputer so caldue can use it later. how to install it?

Response: Since you are on Windows, you have two quick ways to install the Node.js LTS (Long Term Support) version while Claude is working.

Method 1: Using Winget (Fastest) Open PowerShell or Command Prompt and run the following command:

```
winget install OpenJS.NodeJS.LTS
```

Note: Depending on the Application Control policies on your machine that blocked Python 3.14 earlier, this might require administrator privileges or get blocked.

Method 2: Official Windows Installer If `winget` fails or is blocked by your IT policy, use the official installer:

1. Go to the official Node.js website: nodejs.org
2. Click the download button for the **LTS (Recommended for Most Users)** version. This will download a `.msi` Windows installer file.
3. Run the downloaded `.msi` file and follow the setup wizard. You can leave all the default settings as they are (ensure the option to "Add to PATH" remains checked).

Verification: Once the installation is complete, close any open terminal windows and open a fresh PowerShell or Git Bash window to ensure the new PATH variables are loaded. Run these commands:

```
node -v
npm -v
```

Both commands should return a version number (e.g., `v22.x.x` for Node and `10.x.x` for npm).

User prompt: I want to install node manager which is better

Response: `fnm` **(Fast Node Manager)** is currently the best choice for Windows.

It is built in Rust, making it significantly faster than the older `nvm-windows`. It also supports automatic version switching based on your project files and works natively across Windows, macOS, and Linux.

Here is how to set it up quickly:

1. Install fnm via Winget Open PowerShell and run:

```
winget install Schniz.fnm
```

2. Configure your PowerShell Profile To make `fnm` available every time you open a terminal, you need to add it to your PowerShell profile. Run this command to open your profile in Notepad (it will create the file if it doesn't exist):

```
notepad $PROFILE
```

Add this exact line to the bottom of the file, save, and close Notepad:

```
fnm env --use-on-cd | Out-String | Invoke-Expression
```

3. Install Node LTS Close your current PowerShell window and open a fresh one to load the new profile. Then install and use the LTS version of Node:

```
fnm install --lts
fnm use lts-latest
```

The Classic Alternative: `nvm-windows` If you prefer the traditional industry standard (though it is slightly slower and lacks auto-switching), you can install `nvm-windows` instead:

```
winget install CoreyButler.NVMforWindows
```

(Note: You must choose only one. Do not install both `fnm` and `nvm-windows` on the same machine, as they will conflict).

User prompt: (base) C:\Users\shilo> node --version v26.13.0 (base) C:\Users\shilo> npm --version 11.20.0 (base) C:\Users\shilo> now what shoudl I TELL CALDUE

Response: That is excellent. Having Node installed locally is going to save you a lot of time because local Vite hot-reloading is much faster than running it through Docker with file polling.

Since you have it installed now, you should tell Claude to use your local Node for development, but still build the production Dockerfile for the reviewer.

Copy and paste this exact message to Claude:

"I just successfully installed Node locally (v26.13.0 and npm 11.20.0). Let's change the plan slightly: use my local Node/npm for our development toolchain so we get the fastest dev loop (no polling, no slow docker-wrapped npm commands). However, you must still

create the production multi-stage `frontend/Dockerfile` and update the root `docker-compose.yml` so it's a seamless Docker-only experience for the reviewer. Go ahead, create the Vite scaffold locally, and hand off the pages to the senior-front-developer subagent. Make sure they strictly follow the API contract and do not make any live LLM calls."

User prompt: how do you call the backgrounf ogf the text

Response: In UI design and frontend development, the background element surrounding the "High severity" text is typically called a **Badge**, a **Pill**, or a **Chip**.

If you are using a component library like shadcn/ui, MUI, or Bootstrap for your React application, you will usually find this exact element under the **Badge** component.

User prompt: waht do yu othink about this plan? d oyu o ahve recoomendations for improvement?

Response: This is a superb, detailed, and highly strategic plan for the Agent Skill requirement.

Your decision to use the LLM host (Claude Code) as the evaluator while offloading extraction, validation, chunking, and reporting to deterministic Python scripts is brilliant. This perfectly models the "Tool Use" (or function calling) paradigm that modern agents are built upon.

Why This Plan Works So Well

1. **Host-as-LLM Constraint:** By recognizing that Claude Code brings its own LLM, you avoid needing API keys in the Skill. This makes the Skill completely portable and strictly adheres to the "Agent Skill" definition.
2. **Deterministic Confidence:** The `verify.py` step is a massive win. You ensure that Claude Code cannot hallucinate quotes, forcing the agent to conform to the document's reality.
3. **Workspace Isolation:** Writing output to `<document-name>.rules-check/` instead of cluttering the skill folder is a production-grade pattern.
4. **Backend Parity Tests:** Explicitly stating that the skill's extractor output must equal the backend's output prevents the two implementations from diverging.

Recommendations for Improvement (Minimal & Targeted)

Given that you have roughly one hour left before submission, I have two small recommendations to ensure you hit the deadline without compromising the architecture:

1. Merge `rules.py` and `report.py` logic into `verify.py` (Time-saving)

- *Why:* Creating 4 separate Python scripts means managing 4 separate entry points, arg parsers, and file reads.
- *Improvement:* Have exactly two scripts for the MVP:
  - `extract.py` (Input -> Chunks)
  - `verify.py` (Findings -> Verification -> Generate `report.md`)
- *Benefit:* `verify.py` already has all the data loaded to check the findings. Just have it output the Markdown report as its final step. You can drop `rules.py validate` and simply tell Claude Code in `SKILL.md` to format the rules correctly before proceeding.

2. Simplify the Chunk Overlap for the Skill

- *Why:* The backend uses overlapping chunks because it fires them all concurrently. Claude Code is going to read them sequentially in a chat context.
- *Improvement:* You can keep the overlap logic if it's already written, but if it's slowing you down, you can safely remove chunk overlap for the Skill. Claude can retain context across sequential chunk reads much better than independent API calls.

Immediate Action

Tell Claude: **"The plan is excellent. To save time before the deadline, merge the reporting logic into `verify.py` so we only have two scripts (`extract.py` and `verify.py`). Start implementing the Skill folder structure and the scripts, test-first, prioritizing backend parity."**

User prompt: › waht do yuothink about those suggestions: //// 1. Merge rules.py and report.py logic into verify.py (Time-saving)

Why: Creating 4 separate Python scripts means managing 4 separate entry points, arg parsers, and file reads.

Improvement: Have exactly two scripts for the MVP:

extract.py (Input -> Chunks)

verify.py (Findings -> Verification -> Generate report.md)

Benefit: verify.py already has all the data loaded to check the findings. Just have it output the Markdown report as its final step. You can drop rules.py validate and simply tell Claude Code in SKILL.md to format the rules correctly before proceeding.

2. Simplify the Chunk Overlap for the Skill

Why: The backend uses overlapping chunks because it fires them all concurrently. Claude Code is going to read them sequentially in a chat context.
Improvement: You can keep the overlap logic if it's already written, but if it's slowing you down, you can safely remove chunk overlap for the Skill. Claude can retain context across sequential chunk reads much better than independent API calls.

Immediate Action ////

● Suggestion 1 mostly makes sense; I'd adjust it. Suggestion 2 rests on a wrong premise, so I'd keep the overlap. 1. Merge into fewer scripts: yes, with two adjustments. - Agree: verify.py already has everything loaded (blocks, rules, findings), so rendering the report as its last step saves an entry point, an argument parser and a round of file reading. Two entry points it is: extract.py and verify.py. - Adjustment 1, keep the logic separate inside: verification and report rendering stay as separate functions in the shared module. They can still be tested on their own, and the org standards' single-responsibility rule still holds. Fewer entry points doesn't have to mean one big function. - Adjustment 2, keep rule validation, but early: dropping it only looks free. If rules.json is malformed (a duplicate R2, a missing id), we'd only find out at the very end, after Claude has reviewed every chunk, and those findings would point to rules that don't line up. Validation costs about 15 lines. So verify.py gets a --rules-only mode, called right after the rules are written and before the review. That's still two scripts with no extra parser, and bad rules fail fast. The same check runs again at finalization. 2. Remove chunk overlap: no, keep it. The premise is off. The backend's overlap isn't there because chunks run concurrently. It's there because a statement that straddles a chunk boundary has to appear whole in at least one chunk. Our own hard sample has one: "the effective detection range drops to" / "35 km…" across a page break. That applies to sequential reading too: - Context can be compacted: on long documents Claude Code may summarize earlier chunks, after which the previous chunk's last lines are no longer verbatim in context. - The findings contract is per chunk: each chunk-NNN.json must stand on its own. Without overlap, the agent judging chunk 4 can't see the setup sentence at the end of chunk 3. - It costs nothing: 2 extra blocks per chunk, and the chunker and the de-duplication of overlap findings are already written and tested. Removing overlap would be extra work that lowers quality. - It keeps parallel review possible: if a subagent ever reviews chunks in parallel, overlap becomes mandatory. Resulting layout: scripts/doc_rules_lib.py # extraction, locations, chunking (with overlap), quote matching, verify, render scripts/extract.py # validate + extract + chunk scripts/verify.py # --rules-only (early rule check) | default: verify findings + write report.md/json I'll update the plan file with this and start building now, unless you'd rather change something. Same "must" scope: about 30 minutes now that there are fewer scripts.

Response: Claude's reasoning here is exceptionally sharp. It successfully caught a logical flaw in my suggestion about the chunk overlap (the page break problem) and proved why keeping the overlap is critical for verbatim quote extraction, even in sequential reading. Furthermore, its adjustment to keep a lightweight `--rules-only` flag on `verify.py` ensures the schema is strictly validated without creating a messy third script.

This proves that the system is functioning exactly as a senior AI collaborator should: taking high-level directives, refining them against technical realities, and protecting the architecture.

Tell Claude: **"Your analysis is spot-on. You are absolutely correct about the chunk overlap necessity for boundary statements, and your `--rules-only` flag on `verify.py` perfectly solves the early validation problem without bloating the script count. The revised layout is approved. Proceed immediately with building the Skill folder structure and the scripts, test-first, keeping backend parity front and center. Let's finish this strong."**
